

# 影视流媒体用户观看行为排序与推荐系统汇报讲义

## 一、背景与目标

### 1.1 项目背景

随着影视流媒体平台内容规模不断扩大，用户面对的电影数量越来越多。平台如果只是简单展示全部影片，用户很难快速找到自己感兴趣的内容。因此，实际系统通常需要根据用户评分、浏览行为、影片标签和电影类别等信息，对电影进行排序、检索和推荐。

本项目围绕算法课程题目“影视流媒体用户观看行为排序与推荐系统”展开，使用 MovieLens 数据集作为主要演示数据源，模拟一个基础的电影推荐平台。系统需要完成从原始数据读取、数据预处理、电影综合评分计算，到 Top-N 排序、电影查询、相似电影推荐和个性化推荐的完整流程。

从算法与数据结构课程角度看，这个项目不是单纯做一个推荐页面，而是把真实应用场景拆解成几个典型算法问题：

- 如何根据评分、热度和近期行为为电影建立综合排序规则。
- 如何用不同排序算法生成排行榜，并比较运行效率。
- 如何从大量电影信息中快速查询标题、类别或标签。
- 如何利用用户与电影之间的评分关系构建推荐逻辑。
- 如何通过实验数据解释算法复杂度和实际运行时间之间的关系。

因此，项目的核心是把“排序 + 搜索 + 推荐”组合成一个可运行、可演示、可分析的系统。

### 1.2 应用场景

本项目模拟的应用场景是影视流媒体平台的内容推荐模块。用户进入平台后，系统需要支持以下几类需求：

- 展示综合质量较高、评分可信度较强的热门电影。
- 根据电影名称、类型或标签快速查找目标影片。
- 当用户查看某部电影时，推荐内容相似的其他电影。
- 根据用户当前浏览、搜索、喜欢和不喜欢等行为，生成个性化推荐列表。
- 在后台比较不同算法的运行效率，为系统设计提供依据。

这些需求对应到算法实现中，分别涉及排序算法、堆结构、索引结构、相似度计算和协同过滤等内容。

### 1.3 项目目标

本项目第一版的目标是实现一个完整的算法课程设计原型，重点满足以下要求。

首先，完成电影数据的读取与预处理。系统需要从 MovieLens 数据集中读取电影信息、用户评分和标签数据，并将这些原始数据聚合成电影画像，包括平均评分、评分人数、标签数量、近期评分数量、电影类别和综合评分等字段。

其次，实现电影排序与 Top-N 推荐。项目需要至少对比两种排序算法，本系统实现了归并排序和堆排序，并进一步使用 Top-N 堆选择优化排行榜生成过程，用于说明完整排序和只取前 N 个结果之间的效率差异。

第三，实现电影检索功能。系统需要支持按标题、类别和标签查询电影，并对比线性搜索与索引搜索的效率差异。通过倒排索引等数据结构，可以减少重复扫描全部电影数据的开销。

第四，实现基础推荐功能。系统需要根据电影类别、标签和评分结果推荐相似电影，同时结合用户行为记录和历史评分数据，实现个性化 For You 推荐，使推荐结果不只依赖全局热门度。

第五，提供可复现的实验与演示。项目需要能够通过命令行、后端 API 和本地前端页面运行，并输出排序和搜索算法的运行时间对比结果，用实验数据支撑复杂度分析。

### 1.4 本项目的完成范围

当前第一版以 MovieLens small 数据集作为主要汇报和课堂演示路径，已经实现可运行的前端看板、FastAPI 后端接口、命令行实验和单元测试。

Netflix Prize 数据集作为进阶扩展模块，用于展示更大规模评分数据下的工程处理和推荐实验。汇报时可以把 Netflix 部分作为扩展作品介绍，但主线仍然聚焦 MovieLens，因为它更适合完整展示排序、搜索、推荐和性能对比的课程要求。

## 1.5 本部分小结

本项目希望解决的问题可以概括为：在电影数量和用户行为数据不断增加的情况下，如何利用合适的数据结构和算法，让系统能够快速排序、快速查询，并给出更符合用户兴趣的推荐结果。

后续讲义将围绕数据处理、系统架构、排序算法、搜索算法、推荐算法和实验结果展开，说明本项目具体“做了什么、怎么做、为什么这样做”。

# 二、数据集说明

## 2.1 主数据集：MovieLens small

本项目第一版的主要演示数据集是 MovieLens small。该数据集规模适中，字段结构清晰，适合在课堂汇报中完整展示数据读取、预处理、排序、搜索和推荐流程。

当前本地数据路径为：

```
data/ml-latest-small/
```

主要包含以下文件：

| 文件          | 数据量           | 主要作用                                  |
|-------------|---------------|---------------------------------------|
| movies.csv  | 9,742 条电影记录   | 提供电影 ID、电影名和电影类别                      |
| ratings.csv | 100,836 条评分记录 | 提供用户对电影的评分行为                          |
| tags.csv    | 3,683 条标签记录   | 提供用户给电影打上的文本标签                        |
| links.csv   | 9,742 条外部链接记录 | 提供 IMDb、TMDb 等外部平台 ID，本项目第一版不作为核心算法输入 |

其中，movies.csv、ratings.csv 和 tags.csv 是本项目算法流程中最重要的三个文件。

## 2.2 MovieLens 文件格式

movies.csv 用于描述电影基础信息，格式如下：

| 字段      | 含义           | 示例               |
|---------|--------------|------------------|
| movieId | 电影唯一编号       | 1                |
| title   | 电影名称，通常包含年份  | Toy Story (1995) |
| genres  | 电影类别，多个类别用 ` | ` 分隔             |

ratings.csv 用于描述用户评分行为，格式如下：

| 字段        | 含义                   | 示例        |
|-----------|----------------------|-----------|
| userId    | 用户编号                 | 1         |
| movieId   | 被评分电影编号              | 1         |
| rating    | 用户评分，范围通常为 0.5 到 5.0 | 4.0       |
| timestamp | 评分时间戳                | 964982703 |

tags.csv 用于描述用户对电影的文本标签，格式如下：

| 字段        | 含义        | 示例         |
|-----------|-----------|------------|
| userId    | 用户编号      | 2          |
| movieId   | 被标记电影编号   | 60756      |
| tag       | 用户输入的标签文本 | funny      |
| timestamp | 标签创建时间戳   | 1445714994 |

从格式上看，MovieLens 数据可以通过 movieId 把电影信息、评分记录和标签记录关联起来。也就是说，movieId 是本项目进行数据聚合和推荐计算的核心连接字段。

### 2.3 数据在项目中的使用方式

项目并不是直接把原始 CSV 文件展示给用户，而是先把多张表聚合成“电影画像”。每一部电影会被整理成一条综合记录，供后续排序、搜索和推荐模块使用。

聚合后的电影画像大致包含以下信息：

| 字段类型  | 代表字段                            | 用途                  |
|-------|---------------------------------|---------------------|
| 基础信息  | movieId 、 title 、 genres        | 展示电影名称和类别，支持类别搜索    |
| 评分统计  | avg_rating 、 rating_count       | 衡量电影平均评分和评分人数       |
| 可信评分  | bayesian_rating                 | 降低低样本高分电影的偶然性影响     |
| 活跃度信息 | recent_rating_count 、 tag_count | 衡量近期评分和标签活跃程度       |
| 推荐特征  | tags 、 类别列表、标签列表                | 用于相似电影推荐和个性化推荐      |
| 综合排序分 | comprehensive_score             | 用于 Top-N 排行榜和推荐候选排序 |

例如，一部电影如果平均评分很高，但只有极少数用户评分，系统不会直接把它排到最前面，而是会通过可信评分和评分人数进行修正。这样可以避免“一两个人打满分”的电影压过大量用户认可的经典电影。

### 2.4 当前 MovieLens 数据规模

当前第一版演示中，MovieLens 数据规模如下：

| 指标           | 数值      |
|--------------|---------|
| 电影数量         | 9,742   |
| 评分记录数量       | 100,836 |
| 标签记录数量       | 3,683   |
| 可用于排序的电影画像数量 | 9,742   |

这个规模对课堂项目比较合适：数据量足够展示算法效率差异，但又不会让前端演示和本地实验变得过重。

在排序实验中，系统会选取不同规模的数据子集，例如 100、500、1,000、5,000 和 9,742 条电影记录，分别测试归并排序、堆排序和 Top-N 堆选择的运行时间。

在搜索实验中，系统会对比线性扫描和索引搜索。例如按电影名搜索 Toy Story ，按类别搜索 Comedy ，按标签搜索 funny ，观察不同检索结构的效率差异。

### 2.5 扩展数据集：Netflix Prize

除了 MovieLens，本项目还加入了 Netflix Prize 作为进阶扩展数据集。Netflix Prize 数据规模远大于 MovieLens，更适合展示大规模评分数据处理和推荐算法扩展。

当前本地 Netflix 数据主要经过 DuckDB 整理后使用：

```
data/netflix-prize/netflix.duckdb
```

当前本地 Netflix 数据规模如下：

| 表或数据        | 数据量             | 说明              |
|-------------|-----------------|-----------------|
| movies      | 17,770 部电影      | 包含电影 ID、发行年份和标题 |
| ratings     | 100,480,507 条评分 | 包含用户、电影、评分和评分日期 |
| movie_stats | 17,770 条电影统计    | 每部电影的评分人数和平均评分  |
| user_stats  | 480,189 条用户统计   | 每个用户的评分数量和平均评分  |

Netflix 原始数据与 MovieLens 有一个重要区别：Netflix Prize 主要提供电影标题和用户评分，不包含 MovieLens 那样的 genres 和 tags 字段。因此，在本项目中 Netflix 只支持标题检索，不支持类别和标签检索。

汇报时，Netflix 部分可以作为扩展说明：项目不仅能在 MovieLens small 上完成完整算法闭环，也尝试把相同的排序、评分和推荐思想迁移到更大规模的 Netflix 数据中。但第一版主讲仍以 MovieLens 为主，因为它的数据字段更完整，更适合讲清楚排序、搜索和推荐三条主线。

## 2.6 本部分小结

本项目的数据设计可以概括为：用 MovieLens 完成课堂主线，用 Netflix Prize 展示扩展能力。

MovieLens 提供了电影、评分和标签三类核心信息，适合构建综合评分、倒排索引和相似推荐；Netflix Prize 提供更大规模的评分数据，适合验证大数据量下的排序和推荐扩展。后续算法设计都建立在这些数据结构之上。

# 三、评分计算方法

## 3.1 为什么需要综合评分

在电影推荐系统中，如果只按照平均评分排序，会出现一个明显问题：评分人数很少的电影可能因为偶然获得一两个高分而排到最前面。例如，一部电影只有 1 个用户打了 5 分，它的平均分就是 5.0，但这并不一定说明它比几百人都高分评价的经典电影更可靠。

因此，本项目没有直接使用原始平均分作为最终排序依据，而是构建了综合评分。综合评分同时考虑以下因素：

- 评分质量：电影本身评分高不高。
- 评分可信度：评分人数是否足够多。
- 热度：有多少用户参与评分。
- 标签活跃度：用户是否给电影打了较多有意义的标签。
- 近期活跃度：最近一段时间内是否仍然有人评分。

综合评分的目标不是只找“平均分最高”的电影，而是找“评分高、样本相对可靠、用户关注度较高”的电影。

## 3.2 默认评分计算

默认评分使用原始用户评分计算，主要输出字段是：

```
comprehensive_score
```

默认评分分为四个部分：

| 组成部分  | 权重 | 含义              |
|-------|----|-----------------|
| 评分质量分 | 70 | 使用贝叶斯可信评分衡量电影质量 |
| 热度分   | 20 | 根据评分人数计算电影热度    |

| 组成部分  | 权重 | 含义                      |
|-------|----|-------------------------|
| 标签分   | 5  | 根据用户标签数量和标签权重计算标签活跃度    |
| 近期活跃分 | 5  | 根据最近 180 天内的评分数量计算近期活跃度 |

最终公式为：

默认综合评分 = 评分质量分 + 热度分 + 标签分 + 近期活跃分

也就是：

```
comprehensive_score
= rating_score
+ popularity_score
+ tag_score
+ freshness_score
```

### 3.3 贝叶斯可信评分

评分质量分不是直接用平均分计算，而是先计算贝叶斯可信评分。

设：

- R 表示某部电影的平均评分。
- v 表示该电影的评分人数。
- c 表示全体电影的全局平均评分。
- m 表示先验评分人数，本项目 MovieLens 中取 25 。

贝叶斯可信评分公式为：

bayesian\_rating = (v \* R + m \* C) / (v + m)

这个公式可以理解为：当电影评分人数很多时，更相信电影自己的平均分；当电影评分人数很少时，把它向全局平均分拉回去。

评分质量分再由贝叶斯可信评分映射到 70 分：

rating\_score = (bayesian\_rating / 5.0) \* 70

例如，More (1998) 的原始平均分是 5.0 ，但评分人数只有 1 ，经过贝叶斯修正后：

```
bayesian_rating = 3.5592
rating_score = 49.8287
```

它不会因为一个满分评分就排到榜首。

### 3.4 热度、标签和近期活跃分

热度分根据评分人数计算，但使用对数缩放，避免评分人数特别多的电影完全压制其他电影：

popularity\_score = log(1 + rating\_count) / log(1 + max\_rating\_count) \* 20

标签分根据标签证据计算。项目会先对标签做规范化，并根据标签出现次数、标签置信度和标签类型权重得到 tag\_evidence ，再计算：

tag\_score = log(1 + tag\_evidence) / log(1 + max\_tag\_evidence) \* 5

近期活跃分使用数据集中最新评分时间向前推 180 天作为近期窗口，统计每部电影在这个窗口内的评分数量：

```
freshness_score = log(1 + recent_rating_count) / log(1 + max_recent_rating_count) * 5
```

这三个部分分别补充了“有多少人看过或评价”“用户是否留下标签信息”“最近是否仍然活跃”等信息。

### 3.5 用户修订后评分

默认评分会直接使用用户给出的原始分数。但在真实评分数据中，不同用户的打分习惯并不一样：有的用户整体打分偏高，有的用户整体打分偏低。

例如，两个用户都给某部电影打了 4 分，但含义可能不同：

- 如果一个用户平时平均只打 2.5 分，那么 4 分表示他非常喜欢。
- 如果一个用户平时平均打 4.5 分，那么 4 分反而可能只是一般。

为了修正这种“用户打分宽严差异”，项目实现了用户修订后评分，主要输出字段是：

```
preference_adjusted_comprehensive_score
```

对每一条用户评分，先做如下修正：

```
修订后单条评分 = 原始评分 - 该用户平均评分 + 全局平均评分
```

为了避免修订结果超出评分范围，系统还会把结果限制在数据集实际评分范围内：

```
preference_adjusted_rating  
= clip(rating - user_avg_rating + global_avg_rating, min_rating, max_rating)
```

然后，对每部电影计算修订后平均分，再进入同样的贝叶斯修正：

```
preference_adjusted_bayesian_rating  
= (v * preference_adjusted_rating + m * C) / (v + m)
```

最后，把修订后的可信评分映射成 70 分：

```
preference_adjusted_rating_score  
= (preference_adjusted_bayesian_rating / 5.0) * 70
```

用户修订后综合评分为：

```
preference_adjusted_comprehensive_score  
= preference_adjusted_rating_score  
+ popularity_score  
+ tag_score  
+ freshness_score
```

可以看到，默认综合评分和用户修订后综合评分的区别主要在评分质量分。热度分、标签分和近期活跃分保持不变，因为它们描述的是电影整体数据证据，而不是用户打分宽严。

### 3.6 默认评分与修订后评分对比

下面是当前 MovieLens 输出中的几个例子：

| 电影                               | 原始平均分  | 默认可信评分 | 修订后平均分 | 修订后可信评分 | 默认综合评分  | 修订后综合评分 |
|----------------------------------|--------|--------|--------|---------|---------|---------|
| Toy Story (1995)                 | 3.9209 | 3.8772 | 3.8018 | 3.7705  | 77.3101 | 75.8160 |
| Shawshank Redemption, The (1994) | 4.4290 | 4.3612 | 4.2003 | 4.1492  | 86.5684 | 83.6001 |
| More (1998)                      | 5.0000 | 3.5592 | 4.7649 | 3.5501  | 53.9963 | 53.8697 |

从表中可以看到：

- More (1998) 虽然原始平均分是 5.0，但因为只有 1 条评分，可信评分被明显拉低。
- Shawshank Redemption, The (1994) 评分人数多，默认可信评分仍然很高，所以可以稳定排在前列。
- 修订后评分会考虑用户打分习惯，因此部分电影的最终分数会下降或上升。

当前默认 Top-N 排行榜使用：

```
score_mode = default
排序字段 = comprehensive_score
```

用户修订后排行榜使用：

```
score_mode = preference_adjusted
排序字段 = preference_adjusted_comprehensive_score
```

### 3.7 Netflix Prize 评分公式差异

上面介绍的公式是 MovieLens 主演示路径使用的评分公式。Netflix Prize 扩展数据集的评分思想相同，也使用贝叶斯可信评分和用户宽严修正，但由于 Netflix Prize 数据字段不同，具体综合评分公式有两点变化。

第一，Netflix Prize 没有 MovieLens 中的 genres 和 tags 字段。因此 Netflix 评分中没有标签分，也不计算 tag\_score。

第二，Netflix 数据规模更大，评分人数分布也更大，所以先验评分人数设置更高。MovieLens 中  $m = 25$ ，Netflix 中使用：

```
m = 563
```

Netflix 默认综合评分由三部分组成：

| 组成部分  | 权重 | 含义                      |
|-------|----|-------------------------|
| 评分质量分 | 70 | 使用贝叶斯可信评分衡量电影质量         |
| 热度分   | 25 | 根据评分人数计算电影热度            |
| 近期活跃分 | 5  | 根据最近 180 天内的评分数量计算近期活跃度 |

因此 Netflix 默认综合评分公式为：

```
netflix_comprehensive_score
= rating_score
+ popularity_score
+ freshness_score
```

其中评分质量分仍然是：

```
bayesian_rating = (v * R + m * C) / (v + m)
rating_score = (bayesian_rating / 5.0) * 70
```

热度分改为 25 分：

```
popularity_score
= log(1 + rating_count) / log(1 + max_rating_count) * 25
```

近期活跃分仍然是 5 分：

```
freshness_score
= log(1 + recent_rating_count) / log(1 + max_recent_rating_count) * 5
```

Netflix 中的用户宽严修正逻辑与 MovieLens 相同，先对每条评分做用户平均分校正：

```
preference_adjusted_rating
= clip(rating - user_avg_rating + global_avg_rating, min_rating, max_rating)
```

再计算修订后的贝叶斯可信评分：

```
preference_adjusted_bayesian_rating
= (v * preference_adjusted_rating + m * C) / (v + m)
```

最后得到 Netflix 修订后综合评分：

```
netflix_preference_adjusted_comprehensive_score
= preference_adjusted_rating_score
+ popularity_score
+ freshness_score
```

也就是说，MovieLens 与 Netflix 的评分模型可以这样对比：

| 数据集           | 综合评分组成                          | 先验评分人数 | 是否有标签分 |
|---------------|---------------------------------|--------|--------|
| MovieLens     | 评分质量 70 + 热度 20 + 标签 5 + 近期活跃 5 | 25     | 有      |
| Netflix Prize | 评分质量 70 + 热度 25 + 近期活跃 5        | 563    | 无      |

这样处理的原因是：MovieLens 字段更丰富，适合把用户标签作为推荐信号；Netflix Prize 评分数据量更大，但缺少标签和类型信息，所以更强调评分质量、评分人数和近期活跃度。

### 3.8 本部分小结

本项目的评分计算不是简单平均分，而是一个分层评分模型：先用贝叶斯方法解决低样本电影不可靠的问题，再加入热度、标签和近期活跃度形成默认综合评分；在此基础上，再通过用户平均评分修正不同用户的打分宽严差异，得到用户修订后综合评分。

这样设计后，排序结果既能反映电影质量，也能避免样本过少和用户评分习惯带来的偏差。

## 四、排序算法与 Top-N 推荐

### 4.1 排序任务说明

在完成电影画像和综合评分计算后，系统需要根据评分结果对电影进行排序，并生成推荐列表。排序任务主要服务于两个功能：

- 生成全局电影排行榜，例如综合评分最高的前 10 部电影。
- 为推荐模块提供候选集，例如相似电影和个性化推荐中都需要按分数选出更靠前的结果。

本项目中的排序目标可以概括为：

```
按照电影综合评分从高到低排序
```

默认排行榜使用：

```
comprehensive_score
```

用户宽严修正后的排行榜使用：

```
preference_adjusted_comprehensive_score
```

这两个字段的计算方式已经在上一节说明。排序算法本身只关心“用哪个分数字段作为 key”。

## 4.2 实现的排序算法

本项目实现了三种与排序相关的方法：

| 方法        | 项目函数       | 用途          |
|-----------|------------|-------------|
| 归并排序      | merge_sort | 完整排序基线      |
| 堆排序       | heap_sort  | 完整排序基线      |
| Top-N 堆选择 | top_n_heap | 只取前 N 个推荐结果 |

其中，归并排序和堆排序都属于完整排序算法，会把全部电影记录排成一个完整有序列表。Top-N 堆选择则不是完整排序，它只关心前 N 个结果，更符合推荐系统中“取前 10 个”或“取前 20 个”的实际需求。

## 4.3 归并排序

归并排序采用分治思想：

1. 将待排序数组不断拆成左右两半。
2. 分别对左右子数组排序。
3. 将两个有序子数组合并成一个新的有序数组。

在本项目中，归并排序按照 comprehensive\_score 或 preference\_adjusted\_comprehensive\_score 从高到低比较电影。

伪代码可以表示为：

```
merge_sort(items):
    如果 items 长度小于等于 1，直接返回
    将 items 分成 left 和 right
    left_sorted = merge_sort(left)
    right_sorted = merge_sort(right)
    return merge(left_sorted, right_sorted)
```

归并排序的时间复杂度为：

```
O(n log n)
```

它的优点是复杂度稳定，适合作为高效排序算法基线；缺点是需要额外数组空间，并且如果我们只需要前 N 个结果，完整排序会做很多不必要的工作。

## 4.4 堆排序

堆排序利用堆这种数据结构完成排序。堆可以快速找到当前最大值或最小值。

本项目中的堆排序流程是：

1. 根据电影综合评分建立堆结构。
2. 反复取出堆顶元素。

- 3. 调整剩余元素，使其重新满足堆性质。
- 4. 最终得到完整排序结果。

堆排序的时间复杂度也是：

$O(n \log n)$

它的特点是不需要像归并排序那样递归拆分数组，空间使用较少。对于课程项目来说，堆排序也能自然引出后面的 Top-N 堆选择。

### 4.5 Top-N 堆选择

在推荐系统中，很多时候并不需要把全部电影完整排序。例如前端只展示前 10 部电影，那么系统真正需要的是 Top-10，而不是 9,742 部电影的完整有序列表。

因此，本项目额外实现了 Top-N 堆选择：

```
top_n_heap(items, n, key)
```

它的目标是：

只保留评分最高的 **N** 部电影

如果数据总量是 **n**，要取前 **k** 个结果，那么 Top-N 堆选择的典型复杂度可以理解为：

$O(n \log k)$

当 **k** 很小，例如只取前 10 个，而总电影数量很大时， $\log k$  远小于  $\log n$ ，因此 Top-N 堆选择通常比完整排序更适合推荐榜单。

本项目的默认 Top-N 接口就是使用堆选择：

```
GET /api/top?n=10&algorithm=heap
```

在代码中，`algorithm=heap` 对应的是 Top-N 堆选择，而 `algorithm=merge` 对应完整归并排序后再取前 **N** 个。

### 4.6 排序算法复杂度对比

三种方法的理论复杂度如下：

| 方法        | 是否完整排序 | 时间复杂度         | 适用场景            |
|-----------|--------|---------------|-----------------|
| 归并排序      | 是      | $O(n \log n)$ | 需要完整有序列表        |
| 堆排序       | 是      | $O(n \log n)$ | 需要完整有序列表，或讲解堆结构 |
| Top-N 堆选择 | 否      | $O(n \log k)$ | 只需要前 k 个推荐结果    |

其中，**n** 表示电影总数，**k** 表示需要返回的推荐数量。

在本项目的课堂演示中，**k** 通常取 **10**，远小于电影总数，所以 Top-N 堆选择更贴近真实推荐系统的需求。

### 4.7 MovieLens 排序实验结果

MovieLens 当前共有 9,742 部电影。项目分别在不同数据规模下测试归并排序、堆排序和 Top-N 堆选择的运行时间。

实验结果如下：

| 数据规模  | Top-N | 归并排序耗时     | 堆排序耗时      | Top-N 堆选择耗时 |
|-------|-------|------------|------------|-------------|
| 100   | 10    | 0.0001264s | 0.0001797s | 0.0000097s  |
| 500   | 10    | 0.0008163s | 0.0013050s | 0.0000294s  |
| 1,000 | 10    | 0.0018436s | 0.0031164s | 0.0000947s  |
| 5,000 | 10    | 0.0146403s | 0.0215673s | 0.0002527s  |
| 9,742 | 10    | 0.0309048s | 0.0488330s | 0.0005376s  |

从结果可以看出，随着数据规模增大，完整排序耗时增长明显；Top-N 堆选择始终保持很低的耗时。

在 9,742 部电影时：

```
归并排序：0.0309048s
堆排序：0.0488330s
Top-N 堆选择：0.0005376s
```

这说明对于“只展示前 10 部电影”的推荐场景，直接使用 Top-N 堆选择比完整排序更高效。

## 4.8 Netflix 排序实验结果

Netflix Prize 扩展数据集规模更大，当前用于排序的电影数量为 17,770 部。对应实验结果如下：

| 数据规模   | Top-N | 归并排序耗时     | 堆排序耗时      | Top-N 堆选择耗时 |
|--------|-------|------------|------------|-------------|
| 100    | 10    | 0.0001315s | 0.0001939s | 0.0000110s  |
| 500    | 10    | 0.0008632s | 0.0013059s | 0.0000372s  |
| 1,000  | 10    | 0.0020817s | 0.0033915s | 0.0000608s  |
| 5,000  | 10    | 0.0142412s | 0.0220498s | 0.0002874s  |
| 10,000 | 10    | 0.0320733s | 0.0512832s | 0.0004745s  |
| 17,770 | 10    | 0.0644513s | 0.1153770s | 0.0010930s  |

Netflix 数据上的趋势与 MovieLens 一致：完整排序随数据规模增长更明显，而 Top-N 堆选择在只取前 10 个结果时更加高效。

## 4.9 本部分小结

排序模块体现了算法选择与业务需求之间的关系。

如果系统需要完整排行榜，归并排序和堆排序都可以完成任务，时间复杂度都是  $O(n \log n)$ 。但在推荐系统中，前端通常只需要 Top-N 结果，因此没有必要完整排序所有电影。

本项目最终使用 Top-N 堆选择作为默认推荐榜单算法，是因为它更符合“从大量电影中快速选出前几个推荐结果”的实际场景，复杂度也更适合数据规模继续增大的情况。

# 五、搜索功能设计

## 5.1 搜索任务说明

除了排行榜和推荐列表，影视流媒体系统还需要支持用户主动查找电影。用户可能记得电影名称，也可能只知道电影类型或标签。因此，本项目实现了电影搜索功能。

MovieLens 主数据集支持三类搜索：

| 搜索类型 | 查询字段   | 示例                      |
|------|--------|-------------------------|
| 标题搜索 | title  | Toy Story 、 Matrix      |
| 类别搜索 | genres | Comedy 、 Drama 、 Sci-Fi |
| 标签搜索 | tags   | funny 、 mind-bending    |

对应 API 为：

```
GET /api/search?kind=title&query=Toy%20Story
GET /api/search?kind=genre&query=Comedy
GET /api/search?kind=tag&query=funny
```

搜索结果不是简单按匹配顺序返回，而是会结合匹配类型、匹配相似度和电影综合评分进行排序。这样可以让更多相关、更高质量的电影排在前面。

## 5.2 线性搜索

线性搜索是最直接的搜索方法。它会从头到尾遍历所有电影记录，逐条检查是否匹配查询条件。

例如标题线性搜索可以理解为：

```
for movie in movies:
    if query 出现在 movie.title 中:
        加入结果列表
```

类别和标签搜索也类似，只是检查字段从电影标题变成电影类别或标签列表。

线性搜索的时间复杂度为：

$O(n)$

其中  $n$  是电影数量。它的优点是实现简单，不需要额外索引；缺点是数据量变大时，每次查询都要扫描全部电影，效率会下降。

## 5.3 索引搜索

为了提高搜索效率，本项目实现了索引搜索。索引搜索的核心思想是：提前把可查询字段整理成字典或倒排索引，查询时直接根据关键词定位候选电影，而不是每次都扫描全部数据。

项目中主要建立了以下索引：

| 索引          | 作用                       |
|-------------|--------------------------|
| 标题完整短语索引    | 支持完整电影名或标题短语匹配           |
| 标题 token 索引 | 支持按标题中的单词查询              |
| 类别索引        | 支持按 Comedy 、 Drama 等类别查询 |
| 标签索引        | 支持按规范化标签查询               |
| 标签 token 索引 | 支持多词标签的分词查询              |
| n-gram 索引   | 支持模糊搜索和拼写错误召回            |

索引搜索的平均查询效率接近：

$O(1) + O(r)$

其中  $r$  是结果数量。实际运行中还会有候选排序和模糊匹配开销，但整体通常比线性扫描更快。

## 5.4 标题搜索

标题搜索支持多种匹配方式，优先级从高到低大致为：

- 1. 完整短语匹配。
- 2. 标题 token 匹配。
- 3. 子串匹配。
- 4. 模糊匹配。

例如查询：

```
Toy Story
```

可以返回：

```
Toy Story (1995)
Toy Story 2 (1999)
Toy Story 3 (2010)
```

如果用户输入有轻微拼写错误，例如：

```
matrx
```

系统仍然可以通过模糊匹配找到：

```
Matrix, The (1999)
Matrix Reloaded, The (2003)
Matrix Revolutions, The (2003)
```

标题搜索适合用户已经知道电影名称或名称片段的场景。

## 5.5 类别搜索

类别搜索基于 MovieLens 的 genres 字段。每部电影可以属于多个类别，例如：

```
Adventure|Animation|Children|Comedy|Fantasy
```

系统会把类别拆分后建立索引，因此用户可以直接搜索：

```
Comedy
```

当前 MovieLens 数据中，Comedy 可以匹配到 3,756 部电影。结果会按电影综合评分排序，因此排在前面的不是随机电影，而是综合评分更高的喜剧电影。

类别搜索也支持简单模糊匹配。例如用户输入：

```
Comdy
```

系统仍然可以匹配到 Comedy 类别。

类别搜索适合用户没有明确目标电影，只想浏览某类电影的场景。

## 5.6 标签搜索

标签搜索基于 MovieLens 的 tags.csv 。标签是用户给电影留下的文本描述，例如：

```
funny
mind-bending
pixar
time travel
```

标签搜索比类别搜索更细，因为类别通常只有几十种，而标签可以表达更具体的主题、风格或观影感受。

项目在标签搜索前会先做标签规范化。例如：

```
mind bending
mind-bending
mindfuck
```

这些表达可以被归一到相近的标签含义。这样用户输入不同写法时，系统仍然能找到相关电影。

例如查询：

```
mind bending
```

可以返回：

```
Fight Club (1999)
Usual Suspects, The (1995)
Inception (2010)
```

如果用户输入有轻微拼写错误：

```
mind bendng
```

系统仍然可以通过模糊匹配找到相关标签结果。

## 5.7 精确搜索与模糊搜索

本项目的搜索功能不是单一匹配方式，而是把精确搜索和模糊搜索结合起来。

精确搜索包括：

- 标题完整短语完全匹配。
- 标题、类别、标签 key 完全匹配。
- 标题或标签 token 完全匹配。

模糊搜索包括：

- 子串匹配：查询词出现在标题或标签中。
- token 模糊匹配：单词拼写接近时仍能匹配。
- n-gram 模糊匹配：把字符串拆成连续片段，比较查询词与候选词的相似度。
- 编辑相似度匹配：使用字符串相似度判断拼写错误是否仍然接近。

搜索结果排序时，会优先返回更确定的匹配类型。整体优先级可以概括为：

```
完整短语匹配 > 完整 key 匹配 > 完整 token 匹配 > 子串匹配 > 模糊 token 匹配 > 模糊 key 匹配
```

在匹配类型相同的情况下，系统再参考相似度、匹配覆盖度和电影综合评分。

这种设计的好处是：

- 用户输入准确时，可以快速得到精确结果。

- 用户输入不完整或有拼写错误时，系统仍然有一定容错能力。
- 多个结果同时匹配时，综合评分更高的电影会排在更前面。

## 5.8 搜索性能实验

项目对线性搜索和索引搜索进行了运行时间对比。当前 MovieLens 实验结果如下：

| 查询类型 | 查询词       | 线性搜索耗时     | 索引搜索耗时     | 结果数量  |
|------|-----------|------------|------------|-------|
| 标题   | Toy Story | 0.0010660s | 0.0000154s | 3     |
| 类别   | Comedy    | 0.0026210s | 0.0000043s | 3,756 |
| 标签   | funny     | 0.0019635s | 0.0002299s | 25    |

可以看到，索引搜索在三类查询中都明显快于线性搜索。尤其是类别搜索，索引可以直接定位 `Comedy` 对应的电影列表，不需要逐条扫描所有电影。

从复杂度角度看：

| 方法   | 时间复杂度              | 特点             |
|------|--------------------|----------------|
| 线性搜索 | $O(n)$             | 简单，但每次都扫描全部电影  |
| 索引搜索 | 平均接近 $O(1) + O(r)$ | 查询快，但需要提前建立索引  |
| 模糊搜索 | 高于精确索引搜索           | 支持容错，需要额外相似度计算 |

其中 `r` 表示返回结果数量。

## 5.9 Netflix 搜索边界

Netflix Prize 扩展数据集复用了 MovieLens 的标题搜索引擎，但它只支持标题搜索。

原因是 Netflix Prize 数据集中只有电影标题、发行年份和评分记录，没有 MovieLens 中的 `genres` 和 `tags` 字段。因此：

Netflix 支持：标题搜索  
Netflix 不支持：类别搜索、标签搜索

项目没有伪造 Netflix 的类别或标签数据，而是在用户请求 Netflix 的类别或标签搜索时明确返回不支持。这样可以保持数据边界诚实，避免把不存在的字段当成真实功能。

Netflix 标题搜索仍然支持模糊匹配。例如测试中输入：

`matrix`

可以匹配到：

`The Matrix`

### 5.10 本部分小结

搜索模块体现了数据结构对查询效率的影响。线性搜索实现简单，但数据量增长后效率较低；索引搜索通过提前建立字典和倒排索引，显著减少查询时的扫描范围。

本项目在 MovieLens 上实现了标题、类别和标签搜索，并支持精确搜索与模糊搜索；在 Netflix 上则诚实保留标题搜索能力，不扩展不存在的类别和标签字段。这样既满足了课程中“线性搜索与索引搜索对比”的要求，也让系统更接近真实流媒体平台的搜索体验。

## 六、Similar Movies 相似电影推荐

### 6.1 推荐任务说明

相似电影推荐用于解决这样一个问题：当用户正在查看某一部电影时，系统应该推荐哪些内容相近的电影？

例如，用户查看 `Toy Story (1995)` 时，系统希望推荐同样偏动画、冒险、喜剧、儿童或幻想风格的电影；用户查看 `Matrix, The (1999)` 时，系统希望推荐科幻、动作或带有相似标签的电影。

当前版本的 Similar Movies 功能在原有内容相似度基础上，进一步加入了“系列电影识别”。也就是说，系统不仅会看电影类别和标签是否相同，还会识别 `Toy Story`、`The Matrix`、`Harry Potter` 这类同系列电影，使续集、同系列作品能够优先出现在推荐结果中。

### 6.2 推荐流程

MovieLens 相似电影推荐的流程如下：

1. 用户输入一个电影标题，例如 `Toy Story`。
2. 系统先调用标题搜索，找到最匹配的目标电影。
3. 对目标电影和候选电影标题做系列识别，判断是否属于同一系列。
4. 提取目标电影的类别集合和标签集合。
5. 遍历其他电影，计算每部候选电影与目标电影的相似分。
6. 过滤掉既不是同系列、也没有共同类别和共同标签的电影。
7. 使用 Top-N 堆选择返回相似分最高的若干电影。

流程可以表示为：

```
输入电影标题
-> 标题搜索定位目标电影
-> 标题规范化与系列识别
-> 提取目标电影 genres 和 tags
-> 计算每部候选电影的 similarity_score
-> Top-N 堆选择
-> 返回相似电影列表
```

### 6.3 相似度计算公式

MovieLens 相似电影推荐现在使用五个因素：

- 是否属于同一电影系列。
- 共同类别数量。
- 共同标签数量。
- 标题相似度。
- 候选电影自身综合评分。

设：

- `series_score` 表示同系列奖励分。
- `genre_overlap` 表示目标电影和候选电影的共同类别数量。
- `tag_overlap` 表示目标电影和候选电影的共同标签数量。
- `title_similarity` 表示两个标题规范化后的字符串相似度，取值范围大致为 0 到 1。
- `comprehensive_score` 表示候选电影自身综合评分。

相似度公式为：

```
similarity_score
= series_score
+ genre_overlap * 10
+ tag_overlap * 15
+ title_similarity * 8
+ comprehensive_score * 0.1
```

这个公式的含义是：

- 同系列电影获得较大的额外奖励，使续集和同一系列作品优先返回。
- 共同类别说明电影基础类型接近，因此每个共同类别加 10 分。
- 共同标签比类别更细，能表达主题、风格、人物、系列等更具体的信息，因此每个共同标签加 15 分。
- 标题相似度作为补充，用来区分标题接近但不一定完全同系列的候选。
- 候选电影自身综合评分作为质量补充，避免推荐结果只相似但质量过低。

如果一部候选电影既不是同系列，也没有共同类别和共同标签，就不会进入相似电影候选集。

## 6.4 系列电影识别

这次优化中新增了标题系列识别逻辑。系统会先对电影标题进行规范化处理，例如：

- 去掉年份，例如 (1999)。
- 处理英文后置冠词，例如 Matrix, The 会转成 The Matrix 再处理。
- 去掉开头的 the、a、an 等冠词。
- 识别数字、罗马数字、Part、Episode、Chapter 等序列词。
- 识别冒号前的主标题，例如 Lord of the Rings: The Two Towers 的系列 key 是 lord of the rings。

例如：

```
Matrix, The (1999)
Matrix Reloaded, The (2003)
Matrix Revolutions, The (2003)
```

它们会被识别为同一系列：

```
series_key = matrix
```

同系列候选电影的基础奖励为：

```
series_score = 120
```

如果标题被识别为花絮、幕后、bonus material 等补充内容，会降低同系列奖励：

```
series_score = 120 - 35
```

这样可以让真正的正片续集优先于花絮、纪录片、幕后材料等补充内容。

## 6.5 类别、标签和标题相似度

在同系列识别之外，MovieLens 仍然会使用类别和标签判断内容相似性。

MovieLens 的类别字段比较粗，例如 Comedy、Drama、Action、Adventure。很多电影可能共享同一个类别，但实际风格差异很大。

标签字段更细，例如：

pixar  
mind-bending  
time travel  
dark comedy  
superhero

这些标签通常能更准确地表达用户对电影内容和风格的理解。因此在相似度公式中，标签重合的权重 15 高于类别重合的权重 10。

标题相似度则用于补充判断。它不会像同系列识别那样给出很高奖励，而是作为一个较小的加分项：

title\_score = title\_similarity \* 8

这样可以在排序时稍微提高标题相近电影的优先级，但不会让标题相似压过真正的类别、标签和系列关系。

6.6 推荐结果排序

计算出每部候选电影的 similarity\_score 后，系统不需要完整排序全部候选电影，而是直接使用 Top-N 堆选择：

top\_n\_heap(candidates, n, key="similarity\_score")

这样可以快速选出相似分最高的前 N 部电影。这个设计与前面排行榜中的 Top-N 思路一致：推荐系统通常只展示少量结果，因此只取前 N 个比完整排序更合适。

相似电影推荐 API 为：

GET /api/recommend?title=Toy%20Story&n=5

返回结果中会包含：

| 字段                  | 含义           |
|---------------------|--------------|
| similarity_score    | 相似度总分        |
| shared_genres       | 与目标电影共享的类别数量 |
| shared_tags         | 与目标电影共享的标签数量 |
| title_similarity    | 标题规范化后的相似度   |
| series_match        | 是否识别为同一系列    |
| series_key          | 系列识别 key     |
| series_score        | 系列奖励分        |
| comprehensive_score | 候选电影自身综合评分   |

6.7 示例：Toy Story 相似推荐

以 Toy Story (1995) 为目标电影，当前系统返回的部分推荐结果如下：

| 推荐电影                  | 相似分      | 同系列 | 共同类别数 | 共同标签数 | 标题相似度  |
|-----------------------|----------|-----|-------|-------|--------|
| Toy Story 2 (1999)    | 199.5851 | 是   | 5     | 1     | 0.9000 |
| Toy Story 3 (2010)    | 184.3983 | 是   | 5     | 0     | 0.9000 |
| The Lego Movie (2014) | 74.4083  | 否   | 5     | 1     | 0.3158 |
| Bug's Life, A (1998)  | 63.4596  | 否   | 4     | 1     | 0.2105 |

| 推荐电影                  | 相似分     | 同系列 | 共同类别数 | 共同标签数 | 标题相似度  |
|-----------------------|---------|-----|-------|-------|--------|
| Monsters, Inc. (2001) | 60.3617 | 否   | 5     | 0     | 0.3810 |

可以看到，优化后 Toy Story 2 和 Toy Story 3 被识别为同一系列，因此排在最前面。The Lego Movie、Bug's Life 和 Monsters, Inc. 虽然不是同系列，但因为共享较多动画、冒险、儿童、喜剧等类别，也能进入推荐列表。

这比旧版只看类别和标签更符合用户直觉：当用户查看 Toy Story 时，同系列续集应该优先于只是类别相近的其他动画电影。

## 6.8 示例：Matrix 相似推荐

以 Matrix, The (1999) 为目标电影，当前系统返回的部分推荐结果如下：

| 推荐电影                           | 相似分      | 同系列 | 共同类别数 | 共同标签数 | 标题相似度  |
|--------------------------------|----------|-----|-------|-------|--------|
| Matrix Reloaded, The (2003)    | 161.2626 | 是   | 3     | 0     | 0.5714 |
| Matrix Revolutions, The (2003) | 160.3319 | 是   | 3     | 0     | 0.5000 |
| Inception (2010)               | 70.1810  | 否   | 3     | 2     | 0.2667 |
| Terminator, The (1984)         | 56.5283  | 否   | 3     | 1     | 0.5000 |
| Predator (1987)                | 55.1731  | 否   | 3     | 1     | 0.4286 |

这个例子体现了本次优化的重点：Matrix Reloaded 和 Matrix Revolutions 虽然标签数不多，但它们是明确的同系列作品，所以会排在 Inception、Terminator 等内容相似电影之前。

Inception 仍然会进入推荐列表，因为它与 Matrix 共享科幻、动作等类别，并且有共同标签。但在 Similar Movies 场景下，同系列电影的优先级更高。

## 6.9 算法复杂度分析

如果电影数量为  $n$ ，目标电影的类别、标签和系列 key 已经提取完成，那么相似度计算需要遍历所有电影一次。

因此，候选计算复杂度约为：

$$O(n)$$

如果最后取前  $k$  个相似电影，使用 Top-N 堆选择的复杂度约为：

$$O(n \log k)$$

在课堂演示中， $k$  通常是 5 或 10，远小于电影总数，所以这种方法可以在保证逻辑清晰的同时保持较好的运行效率。

系列识别和标题相似度计算会增加少量字符串处理开销，但它们仍然是在遍历候选电影时完成，不改变整体的线性扫描加 Top-N 选择结构。

## 6.10 本部分小结

MovieLens Similar Movies 功能现在是“系列感知 + 内容相似”的推荐逻辑。系列识别负责优先召回续集和同系列作品，类别提供基础相似性，标签提供更细粒度的主题和风格相似性，标题相似度和综合评分作为补充排序信号。

这一部分体现了排序、搜索和推荐之间的连接：系统先通过标题搜索定位目标电影，再进行系列识别和内容相似度计算，最后使用 Top-N 堆选择返回推荐结果。

## 七、MovieLens For You 个性化推荐

### 7.1 功能定位

For You 是本项目中最接近真实流媒体平台“个性化首页”的模块。它和前面的 Top-N 排行榜、Search、Similar Movies 都不一样：

- Top-N 排行榜主要根据全局综合评分排序。
- Search 是用户主动输入关键词查找电影。
- Similar Movies 是围绕某一部目标电影找相似电影。
- For You 会根据当前浏览器会话中的用户行为，动态生成个性化推荐列表。

也就是说，For You 的目标不是回答“全站哪些电影最好”，而是回答：

根据这个用户刚刚搜索、浏览、喜欢和不喜欢的内容，接下来更可能想看什么？

当前 For You 接口为：

```
GET /api/for-you?session_id=...&n=10
```

前端会在用户搜索、查看详情、点击 Like、点击 Dislike、查看 Similar Movies 等行为后记录事件，并刷新 For You 推荐结果。

### 7.2 用户行为数据

For You 使用的不是 MovieLens 原始用户 ID，而是当前浏览器会话的行为记录。每条行为记录大致包含：

| 字段         | 含义                      |
|------------|-------------------------|
| session_id | 当前浏览器会话 ID              |
| event_type | 行为类型                    |
| timestamp  | 行为发生时间                  |
| kind       | 搜索类型，例如标题、类别、标签         |
| query      | 搜索词或 Similar Movies 查询词 |
| movie_id   | 用户浏览、喜欢或不喜欢的电影          |
| source     | 行为来源，例如搜索结果、详情页、推荐列表    |

当前支持的行为类型包括：

| 行为      | 含义            | 基础权重 |
|---------|---------------|------|
| search  | 用户搜索了标题、类别或标签 | 1.0  |
| similar | 用户请求相似电影      | 2.2  |
| view    | 用户打开或查看电影详情   | 2.8  |
| like    | 用户明确喜欢一部电影    | 7.0  |
| dislike | 用户明确不喜欢一部电影   | -7.5 |
| reset   | 清空当前会话偏好      | 0.0  |

可以看到，Like 和 Dislike 的权重最高，因为它们是明确反馈；搜索和浏览权重较低，因为它们只能说明用户可能感兴趣。

### 7.3 电影向量表示

为了根据用户兴趣推荐电影，系统需要先把每部电影表示成一个向量。MovieLens 的电影向量由三类特征组成：

| 特征类型 | 示例                               | 作用                |
|------|----------------------------------|-------------------|
| 类别特征 | genre:action 、 genre:sci-fi      | 表示电影所属类型          |
| 标签特征 | tag:mind-bending 、 tag:pixar     | 表示用户标签中的主题和风格     |
| 数值特征 | num:trusted_rating 、 num:quality | 表示评分、热度、综合质量等数值信息 |

其中，类别特征会按类别数量做缩放，标签特征会结合标签权重和 IDF 思想，较少见但有区分度的标签会更有价值。数值特征包括：

```
avg_rating
bayesian_rating
rating_count
recent_rating_count
comprehensive_score
```

这样处理后，每部电影不再只是一个标题，而是一个包含类别、标签和质量信息的稀疏向量。后续 For You 会把用户兴趣也表示成向量，再用余弦相似度匹配电影。

## 7.4 用户兴趣模型

For You 会把用户行为转化为用户兴趣模型。这个模型不是只保存一个分数，而是同时保存多种信息：

- 用户偏好的类别。
- 用户偏好的标签。
- 用户兴趣向量。
- 用户喜欢过的电影。
- 用户不喜欢的电影。
- 长期兴趣和短期兴趣。
- 明确 Like 带来的正向兴趣。
- Dislike 带来的负向兴趣。

系统最多保留最近 240 条事件作为长期记忆：

```
LONG_EVENT_LIMIT = 240
```

同时只把最近 25 条事件作为短期记忆：

```
SHORT_EVENT_LIMIT = 25
```

长期记忆用于表示用户比较稳定的兴趣，短期记忆用于表示用户最近突然关注的方向。例如用户平时喜欢科幻片，但最近连续搜索了动画电影，那么短期兴趣会提高动画相关电影的权重。

## 7.5 长期兴趣与短期兴趣

系统会在新到旧处理用户行为，并给越新的行为更高权重。

长期权重大致为：

```
long_weight = event_weight * 0.985^index * long_multiplier
```

短期权重大致为：

```
short_weight = event_weight * 0.82^index * short_multiplier
```

其中，index 表示这条事件距离当前有多远，越新的事件 index 越小。0.985 衰减较慢，因此长期兴趣保留时间更长；0.82 衰减较快，因此短期兴趣更强调最近操作。

不同行为还有不同的乘数。例如：

| 行为             | 长期乘数 | 短期乘数 |
|----------------|------|------|
| search         | 0.38 | 1.00 |
| similar        | 0.58 | 1.00 |
| 普通 view        | 0.66 | 1.00 |
| 来自探索区的 view    | 0.12 | 0.45 |
| like / dislike | 1.00 | 0.85 |

这样设计的原因是：搜索和浏览对短期兴趣影响更强，明确喜欢或不喜欢对长期兴趣影响更强；探索区的浏览不一定代表强兴趣，所以权重会低一些。

最终用户兴趣向量由几部分合成：

```
user_vector
= long_vector * 0.55
+ short_vector * short_weight
+ explicit_like_vector * 0.28
- disliked_vector * 0.45
```

其中 short\_weight 不是固定值，而是根据短期信号强度和短期/长期兴趣是否一致动态计算，最高大约到 0.46。如果短期兴趣和长期兴趣方向接近，短期权重会提高；如果短期兴趣只是临时波动，权重会更保守。

## 7.6 搜索行为如何转成兴趣

搜索行为也会进入 For You。

如果用户搜索类别，例如：

```
Comedy
```

系统会增强：

```
genre:comedy
```

如果用户搜索标签，例如：

```
mind-bending
```

系统会增强：

```
tag:mind-bending
```

如果用户搜索标题，例如：

```
Matrix
```

系统会先用标题搜索找到匹配电影，再把匹配电影的类别、标签和向量加入用户兴趣。

搜索结果不是全部等权加入。排在更前面的搜索结果权重更高，大致按排名衰减：

第 rank 个匹配电影的权重 = search\_weight / (rank + 1)

这样可以避免一个模糊查询把太多弱相关电影都当成强兴趣。

## 7.7 Like 与 Dislike 的作用

Like 和 Dislike 是 For You 中最强的信号。

当用户 Like 一部电影时，系统会：

- 把这部电影加入已喜欢集合。
- 把它的类别、标签和电影向量加入明确喜欢向量。
- 提高相似电影和同类电影在推荐中的得分。

当用户 Dislike 一部电影时，系统会：

- 把这部电影加入不喜欢集合。
- 把它的电影向量加入负向兴趣向量。
- 在候选生成时排斥与不喜欢电影过于相似的内容。

系统中有一个不喜欢相似度阈值：

```
DISLIKED_SIMILARITY_LIMIT = 0.72
```

如果候选电影与用户不喜欢的方向太相似，就会被过滤掉。对于没有被直接过滤的候选，也会在得分中扣除不喜欢相似度带来的惩罚。

## 7.8 冷启动推荐

如果用户刚打开页面，还没有足够行为，系统无法可靠判断兴趣。此时 For You 会进入冷启动模式。

判断条件是：

```
signal_strength < 0.3
```

冷启动时，系统不会假装已经了解用户，而是从综合评分较高的电影中选取一组相对多样的结果：

```
engine = cold_start_diverse_high_score
status = cold_start
```

这些结果的推荐桶为：

```
explore
```

也就是说，冷启动推荐主要解决“没有用户画像时先展示什么”的问题。它更接近高质量、多样化的默认推荐。

## 7.9 个性化候选一：Interest

当用户行为足够多后，For You 会进入个性化模式。第一类候选是 Interest，即内容兴趣候选。

系统会用用户兴趣向量和每部电影向量计算余弦相似度：

```
vector_similarity = cosine(user_vector, movie_vector)
```

然后计算个人兴趣分：

```
personal_score
= vector_similarity * 100
+ quality_boost
```

其中：

```
quality_boost = normalized(comprehensive_score) * 18
```

也就是说，Interest 候选既要接近用户兴趣，也要有一定电影质量。只相似但综合质量太低的电影不会自然排到最前。

Interest 推荐结果在前端显示为：

```
Interest
```

这类结果的解释通常类似：

```
Matches your Action, Sci-Fi taste.  
Matches tags you reacted to: Thought-Provoking.
```

## 7.10 个性化候选二：Similar Users

第二类候选是 Similar Users，也就是基于历史 MovieLens 用户评分的协同过滤。

它解决的是另一个问题：只看内容特征可能推荐“看起来像”的电影，但不一定知道“和我品味相近的人还喜欢什么”。协同过滤会利用 MovieLens 里真实用户的评分行为。

系统会先把当前会话行为转成伪评分：

| 当前行为    | 伪评分 | 置信度  |
|---------|-----|------|
| like    | 5.0 | 1.0  |
| dislike | 1.0 | 1.0  |
| view    | 3.6 | 0.35 |

协同过滤至少需要当前用户有 3 部可用电影：

```
MIN_PROFILE_MOVIES = 3
```

然后系统会把当前会话画像与历史 MovieLens 用户做相似度比较。为了减少不同用户打分宽严的影响，历史用户评分会先中心化：

```
centered_rating = rating - user_mean_rating
```

当前会话的伪评分也会以 3.0 为中心：

```
centered_active_rating = pseudo_rating - 3.0
```

用户相似度使用中心化余弦相似度，并乘以共同电影数量的平滑权重：

```
similarity  
= centered_cosine_similarity  
* shared_count / (shared_count + 5)
```

其中历史用户至少要和当前用户共享 2 部电影：

```
MIN_SHARED_MOVIES = 2
```

系统最多保留相似度最高的 24 个历史用户：

```
NEIGHBOR_LIMIT = 24
```

接着，从这些相似用户喜欢的电影中生成候选。候选电影需要满足：

- 当前用户没有已经看过、喜欢过或不喜欢过。
- 相似用户评分至少为 4.0。
- 相似用户对这部电影的中心化评分为正。
- 至少有 2 个相似用户支持。

协同过滤候选的基础分为：

```
collaborative_score
= preference_score * 70
+ support_bonus * 18
+ (avg_neighbor_rating / 5.0) * 12
```

其中：

- `preference_score` 表示相似用户对该电影的加权偏好。
- `support_bonus` 表示有多少相似用户共同支持。
- `avg_neighbor_rating` 表示相似用户对这部电影的平均评分。

进入 For You 混排前，还会加上电影质量分并扣除不喜欢相似度：

```
personal_score
= collaborative_score
+ normalized(comprehensive_score) * 14
- disliked_similarity * 30
```

Similar Users 推荐结果在前端显示为：

```
Similar users
```

解释通常类似：

```
2 similar users rated this around 4.6/5.
```

## 7.11 个性化候选三：Explore

第三类候选是 Explore，用来避免推荐列表过于单一。

如果只根据用户已有兴趣推荐，结果可能全部集中在一个类型里。例如用户喜欢科幻，系统可能一直推荐科幻动作片，缺少探索空间。Explore 会从高综合评分电影中选择一些与用户兴趣有轻微联系、但不完全重复的电影。

Explore 候选分数为：

```
explore_score
= quality_score
+ vector_similarity * 12
- disliked_similarity * 35
```

其中：

```
quality_score = normalized(comprehensive_score) * 100
```

Explore 更重视电影本身质量，同时保留一点与用户兴趣的联系，并强烈避开用户不喜欢的方向。

Explore 推荐结果在前端显示为：

Explore

## 7.12 三类候选混合

For You 最终不会只返回一种候选，而是把 Interest、Similar Users 和 Explore 混合。

如果没有可用协同过滤结果，默认比例大致是：

Interest 约 75%  
Explore 约 25%

如果有协同过滤结果，系统会为 Similar Users 预留位置：

Collaborative share = 25%  
Explore share = 15%  
剩余给 Interest

例如当  $n = 10$  且协同过滤可用时，目标数量通常是：

| 推荐桶           | 目标数量 |
|---------------|------|
| Interest      | 6    |
| Similar users | 2    |
| Explore       | 2    |

系统还会做多样性控制，避免列表被同一种类型刷屏：

- 单一类别最多占大约一半结果。
- 重复类别会扣分。
- 重复标签会扣分。
- 新类别会得到少量奖励。
- Explore 和 Collaborative 会在目标数量内得到展示加成。

最终展示顺序会交错排列，大致是：

2 个 Interest  
1 个 Similar users  
1 个 Explore  
再继续循环

这样用户看到的结果不会全部来自同一种算法来源。

## 7.13 For You 返回信息

For You 接口返回的不只是电影列表，还会返回一些解释信息，方便前端展示和调试。

主要字段包括：

| 字段            | 含义                        |
|---------------|---------------------------|
| status        | cold_start 或 personalized |
| engine        | 当前推荐引擎名称                  |
| bucket_counts | 三类推荐桶各返回多少个               |

| 字段                           | 含义           |
|------------------------------|--------------|
| profile.top_genres           | 当前用户偏好的主要类别  |
| profile.top_tags             | 当前用户偏好的主要标签  |
| profile.liked_movie_count    | 用户明确喜欢的电影数量  |
| profile.disliked_movie_count | 用户明确不喜欢的电影数量 |
| collaborative.neighbor_count | 找到的相似历史用户数量  |
| collaborative.top_similarity | 最高相似用户得分     |

当前个性化模式的引擎名称是：

hybrid\_content\_collaborative\_blend

它表示 For You 是一个混合推荐器，不是单独的内容推荐或单独的协同过滤。

## 7.14 示例：一次个性化推荐结果

假设用户进行了以下行为：

搜索 Sci-Fi  
搜索 mind-bending  
查看并喜欢 Matrix  
喜欢 Inception  
不喜欢 Toy Story  
查看 Fight Club 的相似电影

系统会形成一个偏向科幻、动作、惊悚、哲学和烧脑标签的用户画像。当前代码跑出的一个示例画像为：

| 偏好类型   | 主要结果                                                           |
|--------|----------------------------------------------------------------|
| 主要类别   | action 、 thriller 、 sci-fi 、 drama 、 crime                     |
| 主要标签   | philosophy 、 thought-provoking 、 action 、 complicated 、 sci-fi |
| 喜欢电影数  | 2                                                              |
| 不喜欢电影数 | 1                                                              |
| 推荐桶分布  | 6 个 Interest + 2 个 Similar users + 2 个 Explore                 |

示例推荐结果中可能出现：

| 推荐桶           | 电影                                                        | 推荐原因                                    |
|---------------|-----------------------------------------------------------|-----------------------------------------|
| Interest      | Die Hard (1988)                                           | 匹配 Action、Crime 兴趣                      |
| Interest      | Donnie Darko (2001)                                       | 匹配 Drama、Mystery 和 Thought-Provoking 标签 |
| Similar users | Serenity (2005)                                           | 相似用户给出较高评分                              |
| Explore       | Lord of the Rings: The Fellowship of the Ring, The (2001) | 高分探索，扩展到 Adventure、Fantasy              |
| Interest      | Interstellar (2014)                                       | 匹配 Sci-Fi 和 Thought-Provoking 标签        |

这个例子说明，For You 不会只输出一个方向的电影。它会以用户兴趣为主，同时加入相似用户喜欢的电影和少量高质量探索电影。

## 7.15 算法复杂度分析

For You 的复杂度可以分成几部分看。

第一，构建用户兴趣模型需要遍历最近的行为事件。最多使用 240 条事件：

$$O(e)$$

其中  $e \leq 240$ 。

第二，内容兴趣候选使用倒排向量索引。系统不会对所有电影逐个做完整向量相似度，而是根据用户向量中的特征找到相关电影 postings，再累计点积得分。复杂度可以理解为：

$$O(p)$$

其中  $p$  是用户兴趣特征命中的倒排 posting 数量。

第三，协同过滤需要根据当前用户接触过的电影找到相关历史用户，再计算相似用户和候选电影。因为当前会话中可用电影数量通常很少，所以它更接近“局部召回”，而不是扫描全部用户评分表。

第四，最终混排主要在候选列表窗口中做选择，候选数量被限制在  $n$  的若干倍，例如  $n * 18$  或  $n * 12$ ，因此对前端实时刷新是可接受的。

总体来说，For You 比 Top-N 和 Search 更复杂，但它通过事件数量限制、倒排向量索引、候选数量限制和 Top-N 选择，把实时推荐控制在本地演示可以接受的范围内。

## 7.16 本部分小结

MovieLens For You 是一个混合个性化推荐模块。它先把用户行为转成长期兴趣、短期兴趣、明确喜欢和明确不喜欢，再用电影向量计算内容兴趣候选，同时引入历史评分用户的协同过滤候选，并加入高质量探索候选。

最终结果不是简单按一个分数排序，而是按 Interest、Similar users 和 Explore 三类来源混合展示。这样既能体现用户当前兴趣，也能利用历史用户评分关系，还能保留一定探索性，避免推荐结果过窄。

# 八、MovieLens 标签优化与语义扩展

## 8.1 为什么要优化标签

MovieLens 的 `tags.csv` 来自用户自由输入。自由标签的优点是表达细腻，可以描述电影主题、风格、系列、人物和观影感受；缺点是写法不统一、粒度不一致，也可能出现同义词和拼写差异。

例如，下面这些标签本质上表达的是相近含义：

```
sci fi
scifi
science fiction
science-fiction
```

它们都应该归到：

```
sci-fi
```

再比如：

```
mind bending
mindfuck
mind-bending
```

它们会归到：

mind-bending

如果不做标签优化，系统会把这些写法当作不同标签，导致标签搜索、相似电影推荐和 For You 个性化推荐都变得稀疏、不稳定。

因此，本项目对 MovieLens 标签做了四层处理：

- 标签规范化：统一大小写、符号和空格。
- 标签别名合并：把同义写法映射到同一个 canonical tag。
- 标签权重计算：根据出现次数、置信度和标签类型给标签加权。
- 标签语义扩展：根据标签共同出现的电影学习语义邻居。

## 8.2 标签规范化与别名合并

标签规范化的目标是把用户输入整理成统一格式。处理包括：

- 转为小写。
- 去掉多余空格。
- 把 \_、/ 等符号转为空格。
- 去掉无意义字符。
- 合并重复空白。

规范化后，系统会继续查找别名表，把常见同义词合并。例如：

| 原始标签            | 规范化结果        |
|-----------------|--------------|
| sci fi          | sci-fi       |
| scifi           | sci-fi       |
| science fiction | sci-fi       |
| black humor     | dark comedy  |
| time-travel     | time travel  |
| mindfuck        | mind-bending |
| plot twist      | twist ending |

这个过程的核心作用是减少标签碎片化。例如，如果一部电影被标记为 sci fi，另一部电影被标记为 science fiction，规范化后它们会共享 sci-fi 这个标签，从而提高搜索和推荐的召回能力。

## 8.3 标签类型划分

项目还会把标签划分为不同类型，也就是 tag facet。当前主要有五类：

| 类型        | 含义        | 示例                                     |
|-----------|-----------|----------------------------------------|
| theme     | 主题或内容元素   | aliens 、 time travel 、 crime           |
| style     | 风格或观影感受   | mind-bending 、 dark comedy 、 feel-good |
| franchise | 系列或品牌     | pixar 、 star wars 、 harry potter       |
| person    | 人物、导演或演员  | christopher nolan 、 tom hanks          |
| meta      | 元信息或评价类标签 | oscar 、 classic 、 overrated            |

不同类型的标签权重略有差异：

| 类型        | 权重系数 |
|-----------|------|
| theme     | 1.18 |
| style     | 1.12 |
| franchise | 1.00 |
| person    | 0.86 |
| meta      | 0.66 |

这样设计是因为主题和风格标签通常更能帮助推荐；人物和元信息也有价值，但对内容相似性的解释力相对弱一些。

## 8.4 标签置信度与标签权重

同一个标签出现次数越多，说明它越可靠。但标签次数不能线性放大，否则热门标签会过度支配结果。因此项目使用对数和置信度来计算标签权重。

标签置信度公式为：

$$\begin{aligned} \text{tag\_confidence} \\ &= 0.52 + 0.48 * \min(\log(1 + \text{count}) / \log(1 + 5), 1) \end{aligned}$$

当标签出现次数增加时，置信度会上升，但最高不会超过 1。

标签权重公式为：

$$\begin{aligned} \text{tag\_weight} \\ &= \log(1 + \text{count}) \\ &\quad * \text{tag\_confidence} \\ &\quad * \text{facet\_multiplier} \end{aligned}$$

其中：

- count 是该标签在某部电影上的出现次数。
- tag\_confidence 表示标签证据是否可靠。
- facet\_multiplier 是上一节提到的标签类型权重系数。

例如，当出现次数为 2 时：

| 标签           | 类型    | 置信度    | 权重     |
|--------------|-------|--------|--------|
| sci-fi       | theme | 0.8143 | 1.0556 |
| mind-bending | style | 0.8143 | 1.0020 |
| time travel  | theme | 0.8143 | 1.0556 |

这个设计让标签不只是“有或没有”，而是变成带强度的推荐信号。

## 8.5 标签证据与评分中的 tag\_score

每部电影会保存完整的 tag\_details ，其中包括标签名、显示名、出现次数、类型、置信度和权重。

一部电影的标签证据为：

$$\text{tag\_evidence} = \text{所有规范化标签权重之和}$$

然后在综合评分中计算标签分：

```
tag_score
= log(1 + tag_evidence)
/ log(1 + max_tag_evidence)
* 5
```

例如当前输出中：

| 电影                | tag_evidence | tag_score |
|-------------------|--------------|-----------|
| Toy Story (1995)  | 1.4425       | 0.9812    |
| Fight Club (1999) | 28.9988      | 3.7369    |
| Inception (2010)  | 13.7572      | 2.9574    |

Fight Club 和 Inception 的标签更丰富，因此标签证据更强；Toy Story 标签较少，标签分也较低。

## 8.6 标签搜索中的作用

标签优化直接提升了标签搜索体验。

用户搜索：

```
mind bending
```

系统会先规范化为：

```
mind-bending
```

然后通过标签索引查找相关电影。即使用户输入有轻微拼写错误，例如：

```
mind bendng
```

搜索模块仍然可以通过 token 和 n-gram 模糊匹配找到 mind-bending 相关电影。

也就是说，标签搜索不是简单字符串相等，而是：

- 用户输入
- > 标签规范化
- > 别名合并
- > 索引搜索
- > 必要时模糊匹配

## 8.7 Tag Semantics 标签语义邻居

除了手写别名，项目还实现了标签语义邻居功能：

```
GET /api/tag-semantics?tag=mind-bending&n=8
```

它的目标是找出“经常出现在相似电影上的标签”。例如 mind-bending 可能和 twist ending、philosophy、cerebral 等标签语义接近。

当前语义模型流程如下：

- 根据电影画像构建 tag-movie 权重矩阵。
- 对矩阵做 TF-IDF 加权，降低过于常见标签的影响。
- 使用 LSA 降维，默认维度为 48。
- 对标签向量做归一化。

- 5. 计算标签之间的余弦相似度。
- 6. 保留相似度达到阈值、且共享电影数量足够的语义邻居。

核心参数包括：

| 参数                | 当前值  | 含义            |
|-------------------|------|---------------|
| dimensions        | 48   | LSA 标签向量维度    |
| neighbor_limit    | 10   | 每个标签最多保存多少邻居  |
| min_movie_count   | 3    | 标签至少出现在多少部电影上 |
| min_similarity    | 0.24 | 最低语义相似度       |
| min_shared_movies | 1    | 至少共享多少部电影     |

当前 MovieLens 数据中，语义模型统计结果大致为：

| 指标     | 数值    |
|--------|-------|
| 有效标签数  | 318   |
| 电影数    | 9,742 |
| 语义邻居数量 | 2,030 |
| 模型状态   | ready |

## 8.8 语义邻居示例

以 mind-bending 为例，当前系统找到的前几个语义邻居包括：

| 标签            | 相似度    | 出现电影数 | 共享电影数 |
|---------------|--------|-------|-------|
| twist ending  | 0.7072 | 18    | 7     |
| philosophy    | 0.6407 | 5     | 4     |
| cerebral      | 0.6339 | 7     | 4     |
| dreamlike     | 0.6237 | 11    | 5     |
| psychological | 0.6214 | 10    | 5     |

这说明系统不仅知道 mind-bending 这个字面标签，还能根据电影共现关系推断出它和反转结局、哲学、烧脑、梦境感、心理类标签有较强联系。

再比如 sci-fi 的语义邻居包括：

| 标签                      | 相似度    |
|-------------------------|--------|
| space                   | 0.6259 |
| aliens                  | 0.5847 |
| space opera             | 0.5813 |
| space travel            | 0.5555 |
| artificial intelligence | 0.5339 |

这些结果都符合直觉，因此可以作为前端 Tag Semantics 展示，也可以用于个性化推荐扩展。

## 8.9 For You 中的标签语义扩展

For You 会利用标签语义模型扩展用户兴趣。

当用户画像中出现正向标签时，系统会查找这些标签的语义邻居，并以较小权重加入用户兴趣向量：

```
SEMANTIC_EXPANSION_LIMIT = 5
SEMANTIC_EXPANSION_WEIGHT = 0.3
```

意思是：每个正向标签最多扩展 5 个语义邻居，扩展权重为原标签权重的一部分。

如果用户喜欢 mind-bending 相关电影，系统不只会推荐直接带有 mind-bending 标签的电影，也可能推荐带有 twist ending、cerebral、psychological 等相关标签的电影。

这样可以缓解标签稀疏问题：即使两部电影没有完全相同的标签，只要它们的标签语义接近，也有机会被 For You 召回。

## 8.10 标签优化对各模块的影响

标签优化不是一个孤立功能，而是影响多个模块：

| 模块             | 标签优化的作用                               |
|----------------|---------------------------------------|
| 评分计算           | tag_evidence 和 tag_score 让标签活跃度进入综合评分 |
| 标签搜索           | 规范化、别名和模糊匹配提升搜索召回                     |
| Similar Movies | shared_tags 让相似电影推荐更细粒度               |
| For You        | 标签向量和语义扩展提高个性化兴趣表达                    |
| 前端展示           | Tag Semantics 可以展示标签之间的语义邻居           |

从系统角度看，标签优化让 MovieLens 不只是使用评分数据，还能使用用户留下的文本知识。它把零散的用户标签整理成可搜索、可加权、可扩展的推荐信号。

## 8.11 本部分小结

MovieLens 标签优化可以概括为：

- 自由文本标签
  - > 规范化
  - > 别名合并
  - > 类型划分
  - > 权重计算
  - > 语义邻居学习
  - > 反哺搜索、评分、相似推荐和个性化推荐

这样设计后，系统不再只是机械地匹配标签字符串，而是能够把标签作为一种更稳定、更可解释的推荐特征来使用。

# 九、Netflix For You 个性化推荐与训练加速

## 9.1 Netflix For You 和 MovieLens For You 的区别

MovieLens 和 Netflix 都有 For You 推荐，但两者的实现重点很不一样。

MovieLens 的数据字段更丰富，有电影类别和用户标签，因此它的 For You 更偏内容理解：系统会根据用户搜索、查看、Like、Dislike 等行为，构建类别、标签和质量特征组成的用户兴趣向量，再结合 MovieLens 历史用户协同过滤进行混合推荐。

Netflix Prize 数据集则不同。它的核心数据是大规模用户评分记录，缺少 MovieLens 中的 genres 和 tags 。因此 Netflix For You 不能依赖类别和标签，而是主要依赖：

- 用户评分行为相似性。
- 矩阵分解得到的用户和电影隐向量。
- 电影质量分和热度分。
- 多路候选召回。
- Learning-to-Rank 或神经重排模型。

所以 Netflix For You 现在可以理解为三层实现：

| 层次    | 当前作用                                                                        | 对应代码                                                                                     |
|-------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| 在线模型层 | 前端实时可用的 For You，<br>优先加载离线训练出的<br>stacked hybrid reranker，<br>对多路候选进行真实模型重排 | src/datasets/netflix/recommendation.py 、 src/datasets/netflix/online_reranker.py         |
| 在线回退层 | 当模型 artifact<br>不存在或召回失败时，<br>回退到用户-<br>用户协同过滤推荐                            | src/datasets/netflix/collaborative.py                                                    |
| 离线训练层 | 面向大规模 Netflix<br>数据的候选召回、矩阵分解、<br>Learning-to-Rank<br>和神经重排实验               | matrix_factorization.py 、 candidate_recall.py 、 learning_to_rank.py 、 neural_reranker.py |

汇报时可以先讲在线模型层，说明用户点 Like / Dislike 后，Netflix 如何实时构造候选并用训练好的线性重排模型排序；再讲协同过滤回退层，说明在缺少模型文件时系统仍然可用；最后讲离线训练层，说明这些模型参数和特征权重是怎么训练出来的。

## 9.2 在线 For You 的输入：当前会话行为

Netflix 在线 For You 接口和 MovieLens 使用同一个前端入口：

```
GET /api/for-you?session_id=...&n=10&dataset=netflix
```

前端会把用户行为写入事件记录。Netflix 由于没有类别和标签，所以只接受标题搜索、电影查看、Like、Dislike 和 Similar Movies 等与电影本身相关的行为；如果用户在 Netflix 数据集下请求 genre 或 tag 行为，系统会明确拒绝，因为 Netflix 原始数据没有这些字段。

在线 For You 会把当前会话中的行为转换成一个临时评分画像：

| 行为      | 伪评分 | 置信度  | 含义            |
|---------|-----|------|---------------|
| like    | 5.0 | 1.0  | 用户明确喜欢        |
| dislike | 1.0 | 1.0  | 用户明确不喜欢       |
| view    | 3.6 | 0.35 | 用户看过或点开，弱正向信号 |

对于同一部电影，如果出现多次行为，系统会优先保留置信度更高的行为；如果置信度相同，则保留更新的行为。最后只保留置信度不低于 0.3 的电影作为用户画像。

可以把这一步理解为：

```
浏览器行为
-> 伪评分
-> 去重和置信度筛选
-> active_profile
```

其中 active\_profile 的形式大致是：

```
{
  movie_id_1: 5.0,
  movie_id_2: 1.0,
  movie_id_3: 3.6
}
```

在协同过滤回退路径中，Netflix For You 至少需要 3 部有效电影才能稳定进入个性化推荐：

```
MIN_PROFILE_MOVIES = 3
```

在线 stacked reranker 路径对画像更宽松：即使用户行为较少，也可以先通过热门质量、年份偏好、画像相似和候选回填给出结果；只是当信号不足时，返回状态会更接近冷启动。

### 9.3 用户画像中心化

Netflix 协同过滤不能直接比较原始评分，因为不同用户评分习惯不同。有些用户经常打高分，有些用户比较严格。因此系统会先对当前会话画像做中心化。

在线会话没有真实历史平均分，所以项目使用 3.0 作为临时评分中心：

```
centered_rating = pseudo_rating - 3.0
```

例如：

| 行为      | 伪评分 | 中心化后 |
|---------|-----|------|
| Like    | 5.0 | 2.0  |
| View    | 3.6 | 0.6  |
| Dislike | 1.0 | -2.0 |

然后计算当前画像向量长度：

```
active_norm = sqrt(sum(centered_rating^2))
```

如果 `active_norm = 0`，说明用户画像没有有效方向，系统不会强行做个性化推荐。

### 9.4 寻找相似历史用户

Netflix Prize 的优势是有大量真实用户评分。在线 For You 会把当前会话画像和历史 Netflix 用户进行相似度比较，寻找最相似的一批历史用户。

历史用户评分也会做中心化：

```
历史用户中心化评分 = rating - user_avg_rating
```

对于当前会话用户和某个历史用户，先只看他们共同评价过的电影，计算点积：

```
dot_score
= sum(active_centered_rating * historical_centered_rating)
```

再计算余弦相似度：

```
raw_similarity
= dot_score / (active_norm * historical_user_norm)
```

为了避免“只共同看过很少电影”的用户相似度虚高，系统还加入共同电影数量平滑：

```
overlap_weight
= shared_movie_count / (shared_movie_count + 5)
```

最终相似度为：

```
similarity
= raw_similarity * overlap_weight
```

其中要求：

```
MIN_SHARED_MOVIES = 2
NEIGHBOR_LIMIT = 32
```

也就是说，一个历史用户至少要和当前用户共享 2 部电影，才有资格成为相似用户；系统最多保留相似度最高的 32 个相似用户。

## 9.5 根据相似用户生成候选电影

找到相似用户后，系统会查看这些相似用户喜欢过哪些电影，并把它们作为候选推荐。

候选电影需要满足：

- 当前会话用户还没有看过、喜欢过或不喜欢过。
- 相似用户对这部电影评分至少为 4 分。
- 相似用户对这部电影的中心化评分为正。
- 至少有 2 个相似用户支持。

对应参数是：

```
SUPPORT_LIMIT = 2
```

候选电影的协同过滤分数由三部分组成：

```
preference_score
= weighted_score / similarity_sum

support_bonus
= min(log(1 + support_count) / log(1 + 32), 1)

collaborative_score
= preference_score * 65
+ (avg_neighbor_rating / 5.0) * 20
+ support_bonus * 15
```

三个部分分别表示：

| 部分                  | 权重 | 含义                  |
|---------------------|----|---------------------|
| preference_score    | 65 | 相似用户相对自己平均分有多喜欢这部电影 |
| avg_neighbor_rating | 20 | 相似用户对这部电影的原始平均评分    |
| support_bonus       | 15 | 有多少相似用户共同支持这部电影     |

这样设计后，系统不会只看“某个相似用户喜欢”，而是同时考虑喜欢程度、原始评分和支持人数。

## 9.6 在线 For You 的模型优先与冷启动回退

当前 Netflix 在线 For You 的优先路径是：

```
engine = netflix_stacked_hybrid_reranker
fallback_engine = netflix_user_user_collaborative
```

也就是说，系统会先尝试使用真实在线重排模型。如果模型 artifact 存在，并且候选召回成功，就返回 stacked reranker 的结果；如果模型文件缺失、特征权重不可用或候选池为空，则回退到上一节讲的用户-用户协同过滤。

在线重排模型仍然会保留冷启动回填。冷启动不是随机推荐，而是用 Netflix 综合评分较高的电影做补足：

```
High-scoring Netflix title while your taste profile warms up.
```

回填结果会标记为：

```
recommendation_bucket = top
```

如果重排后的候选结果不足 n 个，系统也会用高综合评分电影补足列表。这样前端始终能展示 For You 内容，同时又不会在用户画像不足时强行假装有很强的个性化信号。

## 9.7 在线 For You 返回结果

Netflix 在线 For You 返回的信息包括：

| 字段                              | 含义                        |
|---------------------------------|---------------------------|
| items                           | 推荐电影列表                    |
| event_count                     | 当前会话累计行为数量                |
| profile.seed_movie_count        | 进入画像的电影数量                 |
| profile.signal_strength         | 当前信号强度，在线版等于有效电影数量        |
| profile.positive_movie_count    | 当前画像中的正向电影数量              |
| profile.negative_movie_count    | 当前画像中的负向电影数量              |
| collaborative.neighbor_count    | 找到的相似历史用户数量               |
| collaborative.top_similarity    | 最高相似用户得分                  |
| collaborative.max_shared_movies | 与相似用户最多共享多少部电影            |
| bucket_counts                   | 各推荐桶返回数量                  |
| candidate_count                 | 进入重排阶段的候选电影数量             |
| route_counts                    | 各召回路由各自产生多少候选             |
| engine                          | 当前引擎名称                    |
| fallback_engine                 | 模型不可用时的回退引擎               |
| model.name                      | 当前加载的离线训练模型名称             |
| model.offline_metric            | 离线实验中的最佳指标摘要              |
| status                          | personalized 或 cold_start |

当 stacked reranker 生效时，单个推荐结果还会包含：

reranker\_score  
recommendation\_sources  
recommendation\_bucket  
recommendation\_reason

其中 recommendation\_sources 会说明这部电影来自哪些召回路线，例如：

popular\_quality  
mf\_user\_top  
profile\_centroid  
item\_item\_cf  
user\_user\_cf  
year\_affinity

推荐桶会根据主要来源归类为：

| 推荐桶           | 含义                |
|---------------|-------------------|
| collaborative | 主要来自相似用户行为        |
| item_item     | 主要来自与种子电影共同被喜欢的电影 |
| profile       | 主要来自用户会话画像或 MF 路由 |
| year          | 主要来自用户偏好的年代范围     |
| top           | 高分冷启动或补足结果        |

解释文案也会对应变化，例如：

Stacked reranker promoted this from similar-user behavior.  
Stacked reranker matched this to your session profile.  
Stacked reranker matched this to your release-year taste.

这说明 Netflix 在线 For You 已经不只是“相似用户也喜欢”，而是把多路召回结果交给训练好的 stacked reranker 统一打分。

### 9.7.1 在线真实重排模型如何工作

在线真实模型由 OnlineNetflixReranker 负责加载和执行。它默认读取：

output/netflix\_residual\_stacked\_10k\_rrf/

关键文件包括：

| 文件                  | 作用                      |
|---------------------|-------------------------|
| feature_weights.csv | 线性重排模型的特征权重、均值和标准差      |
| summary.json        | 训练配置、候选召回参数和离线最佳指标      |
| mf_model.npz        | 保存后的矩阵分解模型，包括用户/电影向量和偏置 |

如果 feature\_weights.csv 或 summary.json 不存在，OnlineNetflixReranker.available 会返回 false，系统自动使用协同过滤回退路径。如果 mf\_model.npz 存在，在线模型会使用真实 MF 电影向量；如果没有，则 mf\_user\_top 路由会降级为画像和质量分的代理路由，并在 model.degraded\_routes 中标出。

在线重排流程如下：

```
当前会话事件
-> active_profile
-> 六路候选召回
-> Weighted RRF 合并候选
-> 构造 25 维用户-电影特征
-> 使用 feature_weights.csv 标准化并线性打分
-> 输出 Top-N
```

六路候选包括：

| 召回路由             | 在线实现                                      |
|------------------|-------------------------------------------|
| popular_quality  | 根据 Netflix 综合评分召回高质量电影                    |
| mf_user_top      | 使用 mf_model.npz 中的电影向量和电影偏置，按当前正向种子构造兴趣向量 |
| profile_centroid | 用用户正向电影的年份和平均评分偏好做画像召回                    |
| item_item_cf     | 在 DuckDB 中查找和种子电影共同被高分喜欢的电影               |
| user_user_cf     | 调用 Netflix 协同过滤模型，查找相似用户喜欢的电影             |
| year_affinity    | 根据用户正向电影的平均年份召回年代接近的电影                    |

候选合并仍然使用 Weighted RRF：

```
combined_score(movie)
= sum(source_weight / (rrf_k + rank_in_source))
```

随后构造和离线 Learning-to-Rank 对齐的特征，例如：

```
mf_score
collaborative_score
quality_score
movie_log_count
profile_similarity
year_affinity
recall_route_score
recall_source_count
recall_user_user_cf
```

最终在线重排分数为：

```
reranker_score
= ((features - mean) / std) dot weights
```

这里的 weights、mean 和 std 都来自离线训练输出的 feature\_weights.csv。因此在线 For You 现在确实使用了真实训练模型，而不是只把离线实验写在报告里。

## 9.8 离线训练版：为什么还需要更复杂的模型

早期在线版本只使用协同过滤，适合前端演示，但 Netflix Prize 数据量很大：评分超过一亿条，用户约 48 万，电影约 1.7 万。仅靠实时用户-用户协同过滤会遇到几个问题：

- 用户数量大，相似用户搜索可能很慢。
- 候选电影多，对每个用户全量打分成本高。
- 只靠协同过滤容易受稀疏性影响。
- 电影没有类别和标签，需要从评分矩阵里学习隐含兴趣结构。

因此项目实现了离线训练版 Netflix 推荐实验，并把其中的 stacked linear hybrid reranker 通过 artifact 接入在线 For You。它不是简单的一个函数，而是一条推荐系统训练流水线：

```
时间切分数据
-> 训练矩阵分解模型
-> 多路候选召回
-> 构造用户-电影特征
-> 训练 Learning-to-Rank / Neural Reranker
-> 在测试集上评估 Top-K 推荐
```

这条流水线更接近真实推荐系统中的“召回 + 排序”架构。离线训练负责学习权重和模型参数，在线 For You 负责加载这些 artifact 并快速推理。

## 9.9 时间切分评估

为了避免数据泄漏，Netflix 离线实验不是随机切分，而是按每个用户的评分时间排序。

基础评估使用两段切分：

```
前 80% 评分 -> train
后 20% 评分 -> test
```

Learning-to-Rank 使用三段切分：

```
早期评分 -> train，用于训练矩阵分解和基础特征
中间评分 -> ranker_train，用于训练重排模型
后期评分 -> test，用于最终评估
```

默认参数是：

```
ranker_ratio = 0.2
test_ratio = 0.2
relevant_threshold = 4.0
top_k = 10
```

也就是说，评分大于等于 4 分的电影被视为用户后续真正喜欢的电影，推荐系统的目标是在 Top-10 里尽量命中这些电影。

评估指标包括：

| 指标               | 含义                     |
|------------------|------------------------|
| precision_at_k   | Top-K 中有多少比例是真正相关电影    |
| recall_at_k      | 用户相关电影中有多少被 Top-K 找回   |
| hit_rate_at_k    | 用户 Top-K 中是否至少命中一部相关电影 |
| map_at_k         | 命中位置越靠前得分越高            |
| catalog_coverage | 推荐覆盖了候选电影集合中的多少不同电影    |
| rmse             | 矩阵分解预测评分的均方根误差         |

## 9.10 矩阵分解模型

Netflix Prize 的经典做法之一就是矩阵分解。项目实现的是带偏置的隐因子模型：

```
predicted_rating(u, i)
    = global_mean
    + user_bias[u]
    + movie_bias[i]
    + user_vector[u] dot movie_vector[i]
```

其中：

- `global_mean` 表示全局平均评分。
- `user_bias` 表示用户整体打分偏高或偏低。
- `movie_bias` 表示电影整体更受欢迎或更不受欢迎。
- `user_vector` 表示用户隐含兴趣向量。
- `movie_vector` 表示电影隐含属性向量。

默认配置为：

| 参数                              | 默认值  |
|---------------------------------|------|
| 隐因子维度 <code>factors</code>      | 48   |
| 训练轮数 <code>epochs</code>        | 20   |
| 学习率 <code>learning_rate</code>  | 0.02 |
| 正则化 <code>regularization</code> | 0.04 |
| batch size                      | 8192 |
| optimizer                       | adam |
| backend                         | auto |
| device                          | auto |

训练目标是让预测评分接近真实评分。对于一条训练样本：

```
error = rating - predicted_rating
```

模型会更新用户偏置、电影偏置、用户向量和电影向量。训练过程中会记录每个 epoch 的 `train_rmse`。

如果本机有 PyTorch 和 CUDA，模型会自动使用 `torch / cuda`；否则回退到 `numpy CPU` 实现。

为了让离线模型可以进入在线 For You，当前矩阵分解模型还支持保存和加载：

```
model.save("mf_model.npz")
MatrixFactorizationModel.load("mf_model.npz")
```

保存内容包括模型配置、全局平均分、用户/电影索引、用户偏置、电影偏置、用户隐向量和电影隐向量。在线 `mf_user_top` 路由会优先读取这个文件，用真实电影向量构造当前会话的兴趣方向。

## 9.11 基础混合排序

矩阵分解能预测用户可能给某部电影打多少分，但单独使用 MF 不一定能在 Top-K 推荐中表现最好。因此项目还实现了基础混合排序，把三类分数加权：

```
hybrid_score
    = mf_weight * mf_score
    + collaborative_weight * collaborative_score
    + quality_weight * quality_score
```

当前默认权重为：

| 分数                  | 权重           |
|---------------------|--------------|
| mf_score            | 0.3551869928 |
| collaborative_score | 0.0841270552 |
| quality_score       | 0.5606859520 |

其中：

- mf\_score 来自矩阵分解预测评分，归一化到 0 到 1。
- collaborative\_score 来自用户-用户协同过滤。
- quality\_score 来自电影质量分，综合贝叶斯评分和热度。

Netflix 离线实验中，电影质量分计算为：

```
bayesian_rating
= (count * avg_rating + 25 * global_avg) / (count + 25)

quality_score
= (bayesian_rating / 5.0) * 0.75
+ log(1 + count) / log(1 + max_count) * 0.25
```

这个混合模型的意义是：既考虑用户个性化兴趣，也保留全局电影质量，避免完全被稀疏个性化信号带偏。

## 9.12 多路候选召回

在 Netflix 这种大数据集上，不能每次都对全量电影做复杂重排。因此项目先用多路召回生成候选池，再对候选池排序。

候选召回来源包括：

| 召回路由             | 含义                |
|------------------|-------------------|
| popular_quality  | 高质量热门电影           |
| mf_user_top      | 矩阵分解给当前用户预测最高的电影  |
| profile_centroid | 用户喜欢电影的隐向量中心附近的电影 |
| item_item_cf     | 与用户喜欢电影共同出现的电影    |
| user_user_cf     | 相似用户喜欢的电影         |
| year_affinity    | 与用户偏好年代接近的电影      |

这些召回来源互相补充：

- popular\_quality 稳定，但个性化弱。
- mf\_user\_top 个性化强，但依赖模型训练效果。
- profile\_centroid 能根据用户喜欢过的电影形成兴趣中心。
- item\_item\_cf 捕捉“喜欢 A 的用户也喜欢 B”。
- user\_user\_cf 利用相似用户行为。
- year\_affinity 利用 Netflix 电影年份信息，捕捉用户年代偏好。

默认候选池大小是：

```
candidate_limit = 1200
```

也就是说，系统先从约一万多部电影中召回约 1200 个候选，再进入后续排序阶段。

## 9.13 Weighted RRF 召回融合

多路召回的难点是：不同召回源的分数尺度不同，不能直接相加。项目实现了 Weighted RRF，也就是加权倒数排名融合。

对于某个召回源中的第 rank 名电影，RRF 分数为：

$$\text{rrf\_score} = \text{source\_weight} / (\text{rrf\_k} + \text{rank})$$

如果一部电影被多个召回源召回，它的总分为各路 RRF 分数之和：

$$\begin{aligned} \text{combined\_score}(\text{movie}) \\ = \text{sum}(\text{source\_weight} / (\text{rrf\_k} + \text{rank\_in\_source})) \end{aligned}$$

这样做的好处是：

- 不要求各路召回分数在同一尺度。
- 排名靠前的电影会得到更高分。
- 被多路召回同时支持的电影会更稳定。

项目还为每个召回源设置了 floor、cap 和 base weight，用来控制候选池结构。以当前 10,000 用户实验为例：

```
candidate_recall_strategy = weighted_rrf
candidate_recall_rrf_k = 20.292280221746303
candidate_limit = 1200
```

部分召回源配置如下：

| 召回源              | floor  | cap    | base_weight |
|------------------|--------|--------|-------------|
| popular_quality  | 0.2589 | 0.6582 | 1.4416      |
| mf_user_top      | 0.4772 | 0.8791 | 0.5578      |
| profile_centroid | 0.1349 | 0.9708 | 0.9014      |
| user_user_cf     | 0.5330 | 0.6841 | 0.8971      |
| year_affinity    | 0.4796 | 0.7449 | 1.7720      |

这些参数不是随便写死的，项目还提供了贝叶斯搜索，用来调节召回策略和权重。

## 9.14 Learning-to-Rank 特征

候选池生成后，系统会为每个用户-电影候选对构造特征。当前 Learning-to-Rank 使用的特征包括：

| 特征                  | 含义                  |
|---------------------|---------------------|
| mf_score            | 矩阵分解预测评分归一化结果       |
| collaborative_score | 用户-用户协同过滤分数         |
| quality_score       | 电影质量分               |
| movie_avg_rating    | 电影平均评分              |
| movie_log_count     | 电影评分人数对数归一化         |
| user_avg_rating     | 用户平均评分              |
| user_log_count      | 用户评分数量对数归一化         |
| profile_similarity  | 候选电影向量与用户正向兴趣中心的相似度 |

| 特征                          | 含义                   |
|-----------------------------|----------------------|
| negative_profile_similarity | 候选电影与用户负向兴趣中心的相似度    |
| signed_profile_similarity   | 正向相似度减去负向相似度后的综合信号   |
| item_item_score             | item-item 协同过滤分数     |
| profile_strength            | 用户正向电影数量是否足够         |
| user_positive_rate          | 用户历史高分比例             |
| movie_year_norm             | 电影年份归一化              |
| movie_has_year              | 是否有年份信息              |
| year_affinity               | 电影年份与用户偏好年份是否接近      |
| recall_*                    | 该电影来自哪些召回路由，以及召回路由分数 |

这一步的核心是把推荐问题转换成监督学习问题：

输入：用户-电影特征向量  
标签：用户后续是否给这部电影打  $\geq 4$  分  
输出：候选电影排序分数

### 9.15 训练样本与负采样

Learning-to-Rank 使用三段切分中的 `ranker_train` 作为监督信号。

正样本定义为：

用户在 `ranker_train` 阶段评分  $\geq 4$  的电影

负样本来自候选池中用户没有喜欢的电影。项目实现了两种负采样：

| 负采样策略         | 含义                               |
|---------------|----------------------------------|
| hybrid_hard   | 选取基础模型分数较高但用户没有喜欢的困难负样本，再混入随机负样本 |
| explicit_hard | 优先使用用户明确低分评价的电影，再用热门度加权抽样补足      |

默认负样本比例为：

`negatives_per_positive = 8`

当前 10,000 用户实验中，训练样本规模为：

| 指标    | 数值        |
|-------|-----------|
| 训练样本数 | 2,217,222 |
| 正样本数  | 246,358   |
| 负样本数  | 1,970,864 |

这说明 Netflix 离线训练不是只在小样本上演示，而是真正构建了百万级用户-电影训练样本。

### 9.16 线性 Learning-to-Rank

项目中的基础重排器是线性模型。它会先标准化特征：

```
x = (features - mean) / std
```

然后计算排序分数：

```
learned_score
    = x dot weights + bias
```

训练目标使用二分类交叉熵，把用户后续高分电影作为正样本：

```
probability = sigmoid(learned_score)

loss
    = -mean(y * log(probability)
            + (1 - y) * log(1 - probability))
      + 0.5 * l2 * ||weights||^2
```

默认训练参数：

| 参数                | 默认值           |
|-------------------|---------------|
| epochs            | 160           |
| learning_rate     | 0.08          |
| l2                | 0.001         |
| negative_sampling | explicit_hard |

训练完成后，模型会输出 `feature_weights.csv`，用于解释哪些特征对排序更重要。现在这个文件也会被在线 For You 读取，用来做真实的 stacked reranker 推理。

最终推荐时，可以直接使用 learned score，也可以和基础 hybrid score 做融合：

```
final_score
    = blend_weight * learned_score
      + (1 - blend_weight) * hybrid_score
```

项目会在验证集上调节 `blend_weight`。当前 10,000 用户实验中：

```
linear_blend_weight = 1.0
linear_blend_mode = value
```

说明在这次实验里，线性重排器本身已经优于继续混入基础 hybrid 分数。

当前这组模型在线加载的部分主要包括：

| artifact            | 在线作用                                                                 |
|---------------------|----------------------------------------------------------------------|
| feature_weights.csv | 提供线性重排器的 <code>weights</code> 、 <code>mean</code> 、 <code>std</code> |
| summary.json        | 提供候选规模、RRF 参数、召回源先验和离线最佳指标                                           |
| mf_model.npz        | 提供矩阵分解电影向量和偏置，用于真实 MF 路由                                             |

### 9.17 残差重排与 MMR 扩展

除了普通线性重排，项目还实现了残差重排实验。残差模型不是从零学习完整排序，而是学习在基础重排器之上的修正量：

```
residual_score
    = base_score + alpha * residual_prediction
```

其中 alpha 会通过验证集调参得到。

当前 10,000 用户实验中：

```
residual_alpha = 0.0
```

这表示残差修正在当前配置下没有带来额外收益，因此系统自动退回到基础线性重排结果。这一点在汇报中可以作为“实验诚实性”的说明：不是每个复杂模型都会提升效果，项目保留了验证机制，而不是强行宣称所有优化都有收益。

项目还预留了 MMR 重排：

```
mmr_score
    = (1 - diversity_weight) * relevance
    - diversity_weight * max_similarity_to_selected
```

MMR 的目标是增加结果多样性，避免 Top-10 全部过于相似。不过当前主要实验结果仍以 stacked linear hybrid reranker 为主。

9.18 神经重排器

项目还实现了一个 PyTorch 神经重排器。它在 Learning-to-Rank 特征之外，额外学习用户 embedding 和电影 embedding。

神经重排器输入包括：

```
user_id
movie_id
features
```

模型结构可以概括为：

```
user_embedding
movie_embedding
normalized_features
-> 拼接
-> MLP
-> ranking_logit
```

默认配置包括：

| 参数                 | 默认值      |
|--------------------|----------|
| embedding_dim      | 24       |
| hidden_dim         | 64       |
| dropout            | 0.10     |
| batch_size         | 2048     |
| epochs             | 25       |
| learning_rate      | 0.001    |
| loss               | pairwise |
| pairs_per_positive | 6        |

神经重排支持两种损失：

| 损失        | 含义             |
|-----------|----------------|
| pointwise | 把候选电影看成正负样本二分类 |
| pairwise  | 让正样本得分高于同用户负样本 |

Pairwise 损失形式可以理解为：

```
loss = softplus(-(positive_logit - negative_logit))
```

如果 GPU 可用，神经重排会使用 CUDA；否则使用 CPU。它是更进一步的实验模块，适合在汇报中作为“深度学习重排扩展”介绍。

## 9.19 训练加速优化一：DuckDB 存储

Netflix Prize 原始评分超过一亿条，如果用普通 CSV 每次扫描，训练和推荐都会很慢。项目先把原始文件导入 DuckDB：

```
data/netflix-prize/netflix.duckdb
```

DuckDB 的作用是：

- 支持大表 SQL 查询。
- 适合本地分析型扫描。
- 比逐行读取文本文件更适合构建统计表和训练样本。
- 可以直接查询 ratings 、 movies 、 movie\_stats 、 user\_stats 等表。

在线协同过滤也使用 DuckDB 查询相似用户和候选电影，避免把全量评分一次性加载到 Python 内存里。

## 9.20 训练加速优化二：预计算 user\_norms

用户-用户协同过滤需要反复计算历史用户中心化评分向量的长度：

```
user_norm
= sqrt(sum((rating - user_avg_rating)^2))
```

如果每次找相似用户都现场计算，会造成大量重复工作。项目检查 DuckDB 中是否存在：

```
user_norms
```

如果存在，就直接读取 rating\_norm ，避免在每次推荐时重新扫描并聚合用户全部评分。

这属于典型的推荐系统工程优化：把不随请求变化的统计量提前算好，把在线计算留给真正和当前用户有关的部分。

## 9.21 训练加速优化三：向量化协同过滤

原始用户-用户协同过滤最慢的地方是寻找相似用户。如果对每个活跃用户都遍历所有可能共享电影的用户，复杂度会很高。

项目实现了向量化协同过滤引擎，把用户评分表示成稀疏矩阵：

```
C = centered rating matrix
M = rating existence mask
```

其中：

- C[u, i] 是用户 u 对电影 i 的中心化评分。
- M[u, i] 表示用户 u 是否评价过电影 i 。

相似用户计算变成两个稀疏矩阵乘法：

```
dot = C[A] @ C.T
```

```
shared = M[A] @ M.T
```

其中  $A$  是当前要评估的一批活跃用户。这样可以一次性计算一批用户和所有历史用户的点积与共同电影数量。

为了控制内存，项目按 block 处理：

```
block_size = 512
```

计算出的结果会写入缓存：

| 缓存                            | 含义                     |
|-------------------------------|------------------------|
| user_neighbor_cache           | 每个用户的相似用户列表            |
| collaborative_raw_cache       | 每个用户对全目录可达电影的原始协同过滤分数  |
| collaborative_raw_array_cache | 同样的分数，但以数组形式保存，方便向量化查找 |

这样后续候选召回、特征构造和评估都能复用同一批协同过滤结果。

## 9.22 训练加速优化四：ANN 近似相似用户搜索

当用户规模达到 Netflix 全量级别时，即使稀疏矩阵乘法也可能遇到  $O(N^2)$  的压力。项目因此加入 ANN 近似近邻搜索。

ANN 路径的思路是：

- 1. 把稀疏中心化评分矩阵投影到低维稠密空间。
- 2. 用 hnswlib 建立 cosine 近似近邻索引。
- 3. 对每个活跃用户先召回一批候选相似用户。
- 4. 再用原始中心化评分公式对候选集做精确重排。

投影默认维度为：

```
ANN_PROJECTION_DIM = 512
```

HNSW 默认参数为：

```
ANN_OVER_FETCH = 20
ANN_EF_CONSTRUCTION = 200
ANN_M = 24
```

自动切换阈值为：

```
ANN_AUTO_MIN_USERS = 50000
```

也就是说，用户数量较小时使用精确稀疏矩阵计算；用户数量很大时自动切换到 ANN，先近似召回，再精确重排。这样既保留了相似度解释，又避免全量用户两两比较。

## 9.23 训练加速优化五：矩阵分解批量打分

矩阵分解在推荐时需要给很多用户和很多候选电影打分。原始做法可能是：

```
for user in users:
    for movie in candidates:
        predict(user, movie)
```

这样会产生大量 Python 循环。项目实现了批量候选打分：

```
candidate_score_matrix(user_ids, movie_ids)
```

它一次性计算：

```
len(user_ids) x len(movie_ids)
```

的预测评分矩阵，核心是矩阵乘法：

```
scores
    = global_mean
    + user_bias[:, None]
    + movie_bias[None, :]
    + user_factors @ movie_factors.T
```

如果模型训练在 CUDA 上，并且环境允许，就使用 GPU GEMM；否则使用 numpy 批量矩阵乘法。

控制开关是：

```
NETFLIX_MF_BATCH = auto | gpu | numpy | off
```

批量打分后，项目还会预热缓存：

```
prime_candidate_score_cache(user_ids, candidate_movie_ids)
```

后续每个用户请求同一个候选池时，只需要从预计算矩阵里按列取值，不必重复做矩阵乘法。

## 9.24 训练加速优化六：特征缓存和数组查找

Learning-to-Rank 需要反复构造用户-候选电影特征。如果每次都重新查字典、重新归一化，会浪费大量时间。

项目使用两类缓存：

第一类是用户候选特征缓存：

```
user_candidate_feature_cache
```

缓存键由三部分组成：

```
(user_id, candidate_fingerprint, exclude_fingerprint)
```

只要用户、候选池和排除列表不变，就可以直接复用特征矩阵。

第二类是电影特征数组查找：

```
movie_feature_lookup_arrays
```

项目会把电影平均分、评分人数、年份、质量分、召回路由分等信息整理成以 `movie_id` 为下标的 numpy 数组。这样构造特征时可以直接批量索引：

```
lookup["quality_score"][movie_ids]
lookup["movie_log_count"][movie_ids]
lookup["recall_route_score"][movie_ids]
```

这比在 Python 中对每个电影做 dict.get 更快，也更适合批量训练。

9.25 训练加速优化七：Top-K 使用 argpartition

在候选池里只需要 Top-10 时，没有必要完整排序全部候选。项目在多个位置使用：

```
np.argpartition
```

先快速选出前 k 个候选位置，再只对这 k 个位置排序：

```
top_positions = np.argpartition(-scores, k - 1)[:k]
ranked_positions = top_positions[np.argsort(-scores[top_positions])]
```

这和前面 Top-N 堆选择的思想一致：推荐系统通常只关心前几个结果，避免完整排序可以明显减少开销。

9.26 当前 Netflix 离线实验结果

当前 output/netflix\_residual\_stacked\_10k\_rrf 中保存了一组 10,000 用户规模的实验结果。配置大致为：

| 参数      | 数值           |
|---------|--------------|
| 用户数     | 10,000       |
| 训练评分数   | 1,316,740    |
| 测试评分数   | 438,909      |
| 电影数     | 15,626       |
| 候选电影数   | 10,565       |
| Top-K   | 10           |
| 相关阈值    | 评分 >= 4      |
| 训练样本数   | 2,217,222    |
| MF 模型文件 | mf_model.npz |

主要算法结果如下：

| 算法                             | Precision@10 | Recall@10 | HitRate@10 | MAP@10   | 覆盖率      |
|--------------------------------|--------------|-----------|------------|----------|----------|
| Popular Quality                | 0.074112     | 0.024215  | 0.307436   | 0.045325 | 0.007478 |
| User-User CF                   | 0.029713     | 0.017375  | 0.223835   | 0.010972 | 0.303171 |
| Matrix Factorization           | 0.008912     | 0.003754  | 0.076176   | 0.003675 | 0.894179 |
| Hybrid MF + CF + Quality       | 0.080880     | 0.033912  | 0.385129   | 0.045146 | 0.094841 |
| Stacked Linear Hybrid Reranker | 0.102468     | 0.051382  | 0.477188   | 0.058697 | 0.068434 |
| Residual Hybrid Reranker       | 0.102468     | 0.051382  | 0.477188   | 0.058697 | 0.068434 |

从结果可以看出：

- 单独热门质量基线已经很强，说明 Netflix 中全局热门和质量是重要信号。
- 单独 MF 在这组 Top-K 指标上不如热门基线，但覆盖率很高，说明它能探索更多电影。

- Hybrid 把 MF、协同过滤和质量分结合后，明显优于单一路径。
- Stacked Linear Hybrid Reranker 进一步提升 Precision@10、Recall@10、HitRate@10 和 MAP@10，是当前这组实验中的最好结果。
- Residual Reranker 与线性重排结果相同，因为本次验证中 `residual_alpha = 0.0`，系统没有采用无收益的残差修正。

这组实验输出现在不仅用于报告分析，也作为在线 Netflix For You 的模型 artifact 来源。在线路径会读取同目录下的 `summary.json`、`feature_weights.csv` 和 `mf_model.npz`，把离线训练结果转化为实时推荐排序。

## 9.27 本部分小结

Netflix For You 的核心是从大规模评分行为中学习用户偏好。

在线演示层现在优先使用 stacked hybrid reranker：把当前会话行为转成伪评分，走热门质量、MF、画像、item-item、user-user 和年份偏好等多路召回，再用离线训练得到的线性权重进行重排；如果模型 artifact 不可用或候选召回失败，则回退到用户-用户协同过滤。

离线训练层进一步扩展为完整的“召回 + 排序”系统：先训练带偏置矩阵分解模型，再用热门质量、MF 用户 Top、兴趣中心、item-item CF、user-user CF 和年份偏好做多路候选召回，最后用 Learning-to-Rank 或神经重排器学习候选排序。当前在线接入的是 stacked linear hybrid reranker。

为了让这条路径能在 Netflix Prize 规模上跑起来，项目还实现了 DuckDB 存储、user\_norms 预计算、稀疏矩阵向量化协同过滤、ANN 近似近邻、MF 批量打分、候选特征缓存、数组查找和 argpartition Top-K 等一系列加速优化。

因此，Netflix 部分不仅是把 MovieLens 算法搬到更大数据集上，而是展示了推荐系统在数据规模变大后如何从简单实时推荐升级到离线训练、候选召回、模型重排、在线推理和工程加速。

# 十、Netflix Similar Movies 相似电影推荐

## 10.1 Netflix 相似推荐的核心思路

Netflix 的 Similar Movies 和 MovieLens 的 Similar Movies 差别很大。

MovieLens 有 genres 和 tags，所以可以用类别重叠、标签重叠、标题相似度和综合评分一起计算相似度。Netflix Prize 数据集没有类别和标签，只有电影标题、年份和大量用户评分。因此 Netflix 相似电影推荐不能使用 genre/tag overlap，而是主要依赖两类信号：

- 用户行为信号：哪些电影被相似用户共同高分评价。
- 标题系列信号：哪些电影属于同一个系列，例如 `The Matrix`、`The Matrix: Reloaded`。

所以 Netflix Similar Movies 可以概括为：

```
目标电影
-> 当作一个强喜欢种子
-> 运行 Netflix For You 的用户-用户协同过滤
-> 得到协同过滤候选
-> 再合并同名系列候选
-> 按系列优先和相似分排序
```

也就是说，它确实是在 Netflix For You 的协同过滤模型上，加了一层同名系列搜索。

## 10.2 API 入口与整体流程

Netflix 相似推荐接口仍然复用通用推荐接口：

```
GET /api/recommend?title=The%20Matrix&n=10&dataset=netflix
```

后端返回的引擎名称为：

```
engine = netflix_collaborative_similarity
```

整体流程如下：

```
用户输入 title_query
-> Netflix 标题索引搜索
-> 取最匹配电影作为 target
-> 用 target 构造单电影强喜欢画像
-> 用户-用户协同过滤召回候选
-> 同系列标题匹配召回候选
-> 合并候选并排序
-> 返回 Top-N
```

如果标题搜索找不到目标电影，接口会返回：

```
target = None
items = []
```

## 10.3 第一步：标题搜索定位目标电影

Netflix 相似推荐首先要将用户输入的标题定位到一部目标电影。

它调用 Netflix 标题搜索引擎：

```
matches = engine.index_title_search(title_query)
target = matches[0]
```

Netflix 标题搜索继承了前面讲过的标题索引思想，支持标题规范化、token 搜索和模糊匹配。但由于 Netflix 没有类型和标签，这里只搜索标题。

例如用户输入：

```
Matrix
```

可能匹配到：

```
The Matrix
```

后续相似推荐就以 The Matrix 作为 target。

## 10.4 第二步：把目标电影变成 For You 种子

定位目标电影后，系统不会用 genre/tag 计算相似度，而是把目标电影当成一个“用户非常喜欢的电影”。

代码中构造的临时画像是：

```
active_profile = {
    target_movie_id: 5.0
}
```

然后调用 Netflix 协同过滤模型：

```
recommend_for_profile(
    {target_movie_id: 5.0},
    exclude_movie_ids={target_movie_id},
    limit=max(n * 8, 40),
    min_profile_movies=1,
    min_shared_movies=1,
    support_limit=2
)
```

这里和普通 For You 有几个差别：

| 参数                 | 普通 Netflix For You | Similar Movies |
|--------------------|--------------------|----------------|
| min_profile_movies | 3                  | 1              |
| min_shared_movies  | 2                  | 1              |
| support_limit      | 2                  | 2              |
| 画像来源               | 用户多个行为             | 目标电影一部         |

为什么 Similar Movies 可以把门槛降到 1？因为它不是判断一个真实用户的完整兴趣，而是在回答“喜欢这部电影的人还喜欢什么”。只用目标电影作为种子是合理的。

## 10.5 协同过滤相似候选

协同过滤部分沿用 Netflix For You 的相似用户逻辑。

对于目标电影，系统会找出历史上也评价过这部电影、并且评分方向相近的用户。因为目标画像中只有一部电影，所以相似度可以理解为：

```
喜欢 target 的历史用户
-> 他们还高分评价了哪些电影
```

候选电影仍然需要满足：

- 不能是目标电影本身。
- 相似用户评分至少为 4 分。
- 相似用户对该电影的中心化评分为正。
- 至少有 2 个相似用户支持。

协同过滤候选的分数沿用上一章 For You 中的公式：

```
preference_score
= weighted_score / similarity_sum

support_bonus
= min(log(1 + support_count) / log(1 + 32), 1)

collaborative_score
= preference_score * 65
+ (avg_neighbor_rating / 5.0) * 20
+ support_bonus * 15
```

对 Similar Movies 来说，协同过滤候选会被包装成：

```
similarity_score = collaborative_score
recommendation_bucket = collaborative
```

推荐原因会写成：

```
Users who strongly align with target_title also rated this highly.
```

也就是说，Netflix 的非系列相似电影，本质上是“喜欢目标电影的相似用户也高分喜欢的电影”。

## 10.6 第三步：同名系列候选

只靠协同过滤有一个问题：系列电影不一定总能稳定排到最前。比如用户搜索 `The Matrix`，直觉上 `The Matrix: Reloaded` 和 `The Matrix: Revolutions` 应该优先出现，即使它们的协同过滤分不是最高。

因此项目加入了同名系列识别。

系列识别复用统一的 `series_match` 逻辑，会处理：

- 去掉标题末尾年份。
- 把 `Matrix`, `The` 转成 `The Matrix`。
- 去掉开头冠词 `The`、`A`、`An`。
- 识别冒号前的系列核心。
- 识别数字、罗马数字、`Part`、`Episode` 等续集标记。
- 排除过弱或误匹配标题。

例如：

| 标题                      | series key |
|-------------------------|------------|
| The Matrix              | matrix     |
| The Matrix: Reloaded    | matrix     |
| The Matrix: Revolutions | matrix     |

而下面这种标题不会误判为同系列：

The Matrix  
Sexual Matrix

测试中也明确验证了 `Sexual Matrix` 不会被推荐进 `Matrix` 系列结果。

## 10.7 系列候选评分公式

同系列候选有一套单独的相似度公式：

```
series_score = 180
```

如果候选标题像 `bonus material`、`extras`、`behind scenes` 这类补充内容，会扣分：

```
series_score = 180 - 45
```

最终系列相似分为：

```
series_similarity_score
= series_score
+ title_similarity * 10
+ comprehensive_score * 0.1
```

其中：

| 项                   | 含义                  |
|---------------------|---------------------|
| series_score        | 同系列强奖励              |
| title_similarity    | 标题规范化后的字符串相似度       |
| comprehensive_score | Netflix 综合评分，作为质量补充 |

当前常量为：

```
SERIES_MATCH_BONUS = 180
SUPPLEMENTAL_SERIES_PENALTY = 45
TITLE_SIMILARITY_WEIGHT = 10
```

这个分数设计非常明确：同系列关系优先级很高，标题相似度和电影质量只是在同系列候选内部进一步排序。

## 10.8 协同过滤和系列候选的合并

Netflix Similar Movies 会先得到协同过滤候选，再扫描全体 Netflix 电影找同系列候选，最后把两类候选合并。

如果某部电影同时出现在协同过滤候选和同系列候选中，说明它既是同系列，又被相似用户高分支持。这时系统会把协同过滤分作为额外加分：

```
final_similarity_score
    = series_similarity_score
    + collaborative_score * 0.25
```

当前融合权重为：

```
COLLABORATIVE_BLEND_WEIGHT = 0.25
```

推荐原因也会改成：

```
Same series as target_title; similar users also rated it highly.
```

如果某部电影只来自系列匹配，则推荐桶是：

```
recommendation_bucket = series
```

如果只来自协同过滤，则推荐桶保持：

```
recommendation_bucket = collaborative
```

## 10.9 最终排序规则

合并完成后，Netflix Similar Movies 的排序 key 是：

```
(
    是否同系列,
    similarity_score,
    comprehensive_score
)
```

按降序排列。

也就是说：

1. 同系列电影优先于普通协同过滤电影。
2. 同为系列电影时，看 `similarity_score`。
3. 分数仍接近时，再看 Netflix 综合评分。

这个排序规则保证了用户搜索一部系列电影时，续集和同系列作品会稳定排在前面；但如果某部同系列电影同时被协同过滤支持，它还会得到额外加分，排序会更靠前。

## 10.10 示例：Matrix 系列

测试中构造了一个例子：

```
target = The Matrix
候选包含：
  The Matrix: Reloaded
  The Matrix: Revolutions
  Sexual Matrix
```

推荐结果要求：

```
Top 1 = The Matrix: Reloaded
Top 2 = The Matrix: Revolutions
Sexual Matrix 不进入结果
```

这说明系列识别不是简单包含 Matrix 字符串，而是会判断标题结构和系列 key。

对于 The Matrix: Reloaded ，结果字段中会包含：

```
series_match = true
series_key = matrix
recommendation_bucket = series
recommendation_reason = Same series as The Matrix.
```

如果它同时也被协同过滤召回，则原因会变成“同系列 + 相似用户也高分评价”。

## 10.11 与 MovieLens Similar Movies 的对比

MovieLens 和 Netflix 的 Similar Movies 可以这样对比：

| 对比项       | MovieLens                                    | Netflix                        |
|-----------|----------------------------------------------|--------------------------------|
| 数据字段      | title、genres、tags、ratings                    | title、year、ratings             |
| 主相似信号     | genre overlap、tag overlap、series、title、score | user-user collaborative、series |
| 是否使用标签    | 使用                                           | 不使用                            |
| 是否使用类型    | 使用                                           | 不使用                            |
| 系列匹配作用    | 提升同系列电影排序                                    | 强规则，系列电影优先                     |
| 非系列相似电影来源 | 内容相似和标签相似                                    | 喜欢目标电影的相似用户还喜欢什么               |

Netflix 这样设计是由数据决定的：既然没有类别和标签，就不能伪造内容相似度；更合理的做法是利用 Netflix Prize 最强的资产，也就是大规模用户评分行为。

## 10.12 本部分小结

Netflix Similar Movies 可以总结为：

```
标题搜索定位目标电影
-> 目标电影作为 5 分喜欢种子
-> 调用 Netflix For You 的用户-用户协同过滤
-> 找到“喜欢这部电影的人还喜欢什么”
-> 扫描并加入同名系列电影
-> 同系列优先，协同过滤分辅助加权
```

这一部分和前面的 Netflix For You 是连在一起的：For You 是从多个用户行为构建画像，Similar Movies 则是把“当前目标电影”当成一个极简画像。两者共用协同过滤能力，只是 Similar Movies 额外加入了同名系列搜索，使结果更符合用户对电影系列的直觉。

# 十一、系统整体架构与 API 流程

## 11.1 系统整体结构

本项目不是只实现单个算法函数，而是做成了一个可以运行、可以演示、可以切换数据集的推荐系统实验平台。

整体结构可以分为五层：

```
前端页面 web/
-> FastAPI 后端 src/api.py
-> 数据集注册与统一 pipeline src/core/
-> MovieLens / Netflix 独立算法模块 src/datasets/
-> 本地数据与实验输出 data/、output/
```

从用户角度看，打开网页后可以完成：

- 查看数据集摘要。
- 查看 Top-N 排行榜。
- 搜索电影。
- 查看相似电影推荐。
- 点击 Like / Dislike / View 形成行为记录。
- 刷新 For You 个性化推荐。

从后端角度看，系统会根据 dataset=movielens 或 dataset=netflix 选择不同的数据集服务。MovieLens 和 Netflix 共用同一套 API 入口，但内部算法不同。

## 11.2 前端展示层

前端位于：

```
web/
```

它主要负责交互展示，不直接实现核心算法。前端会调用后端 API 获取结果，并把用户操作记录成事件。

前端主要区域包括：

| 区域             | 作用                 |
|----------------|--------------------|
| Dashboard      | 展示数据集规模、运行时间和摘要    |
| Top-N          | 展示综合评分排行榜          |
| Search         | 支持标题、类型、标签搜索       |
| Similar Movies | 输入标题后展示相似电影        |
| For You        | 根据当前会话行为动态推荐       |
| Tag Semantics  | MovieLens 标签语义邻居展示 |

当用户点击电影、Like、Dislike 或 Similar Movies 时，前端会调用：

```
POST /api/events
```

这些事件会进入当前浏览器会话的行为记录，随后影响 For You 推荐。

# 11.3 后端 API 层

后端入口是：

```
src/api.py
```

启动后，FastAPI 和静态前端在同一个端口提供服务：

```
http://127.0.0.1:8013/  
http://127.0.0.1:8013/docs
```

主要 API 如下：

| API                 | 作用               |
|---------------------|------------------|
| GET /api/health     | 检查服务状态和可用数据集     |
| GET /api/dashboard  | 返回数据集摘要和实验运行时间   |
| GET /api/top        | 返回 Top-N 排行榜     |
| GET /api/search     | 搜索电影             |
| GET /api/recommend  | 相似电影推荐           |
| POST /api/events    | 记录用户行为           |
| GET /api/for-you    | 个性化 For You 推荐   |
| GET /api/tag-antics | MovieLens 标签语义邻居 |

这些 API 都支持数据集参数：

```
dataset=movielens  
dataset=netflix
```

例如：

```
GET /api/top?n=10&algorithm=heap&dataset=movielens  
GET /api/search?kind=title&query=Matrix&dataset=netflix  
GET /api/for-you?session_id=...&dataset=movielens
```

# 11.4 数据集服务层

后端内部有两个服务类：

| 服务                  | 数据集           | 作用                                   |
|---------------------|---------------|--------------------------------------|
| MovieLensApiService | MovieLens     | 加载电影画像、搜索引擎、标签语义模型、For You 模型        |
| NetflixApiService   | Netflix Prize | 加载 Netflix 评分结果、标题搜索引擎、协同过滤模型和在线重排模型 |

MovieLens 服务加载：

```
movies.csv  
ratings.csv  
tags.csv
```

并构建：

- 电影画像 profiles
- 标题/类型/标签索引
- 标签语义模型
- 电影向量模型
- 历史用户协同过滤模型

Netflix 服务加载：

```
output/netflix/movie_scores.csv
data/netflix-prize/netflix.duckdb
output/netflix_residual_stacked_10k_rmf/
```

并构建：

- Netflix 电影评分表
- 标题搜索索引
- DuckDB-backed 用户-用户协同过滤模型
- Online stacked hybrid reranker
- 离线训练 artifact：summary.json、feature\_weights.csv、mf\_model.npz

这样设计后，前端不需要知道 MovieLens 和 Netflix 的内部差异，只需要带上 `dataset` 参数即可。

## 11.5 数据集边界处理

项目在双数据集设计中有一个重要原则：不伪造数据字段。

MovieLens 有：

```
title
genres
tags
ratings
```

因此 MovieLens 支持标题搜索、类型搜索、标签搜索、标签语义、内容相似推荐和混合 For You。

Netflix 有：

```
title
release_year
ratings
```

因此 Netflix 支持标题搜索、评分排序、协同过滤、矩阵分解和训练重排，但不支持真实的类型和标签搜索。

如果用户在 Netflix 下请求：

```
kind=genre
kind=tag
```

系统不会返回假结果，而是明确说明 Netflix 数据集没有这些字段。这个边界处理让系统更可信，也能在汇报中体现数据驱动设计。

## 11.6 统一运行方式

项目提供了本地一键演示脚本。

Windows 下运行：

```
.\start_frontend.ps1
```

也可以指定数据集：

```
.\start_frontend.ps1 -Dataset movielens
.\start_frontend.ps1 -Dataset netflix -Port 8014
```

命令行实验入口为：

```
.\run.ps1 demo
.\run.ps1 top -n 10 --algorithm heap
.\run.ps1 search title "Toy Story"
.\run.ps1 recommend "Toy Story"
```

Netflix 扩展实验入口为：

```
.\run.ps1 --dataset netflix demo
.\run.ps1 --dataset netflix top -n 10 --algorithm heap
.\run.ps1 --dataset netflix search title "Matrix"
.\run.ps1 --dataset netflix recommend "The Matrix"
```

更复杂的 Netflix 离线训练实验包括：

```
.\run.ps1 --dataset netflix large-search --profile smoke
.\run.ps1 --dataset netflix bayes-search --profile smoke --mode hybrid
.\run.ps1 --dataset netflix rerank --profile smoke
.\run.ps1 --dataset netflix neural-rerank --profile smoke
```

## 11.7 输出文件与实验结果

MovieLens 输出主要在：

```
output/movielens/
```

常见文件包括：

| 文件                  | 含义       |
|---------------------|----------|
| movie_profiles.csv  | 聚合后的电影画像 |
| sorting_runtime.csv | 排序算法运行时间 |
| search_runtime.csv  | 搜索算法运行时间 |
| tag_semantics.json  | 标签语义邻居缓存 |
| user_events.jsonl   | 前端会话行为记录 |

Netflix 输出主要在：

```
output/netflix/
output/netflix_residual_stacked_10k_rrf/
```

常见文件包括：

| 文件                  | 含义             |
|---------------------|----------------|
| movie_scores.csv    | Netflix 电影综合评分 |
| sorting_runtime.csv | Netflix 排序运行时间 |

| 文件                   | 含义                            |
|----------------------|-------------------------------|
| metrics.csv          | 推荐模型评估指标                      |
| candidate_recall.csv | 多路候选召回分析                      |
| mf_training.csv      | 矩阵分解训练曲线                      |
| mf_model.npz         | 在线 For You 使用的矩阵分解模型 artifact |
| ranker_training.csv  | Learning-to-Rank 训练曲线         |
| feature_weights.csv  | 线性重排特征权重                      |
| summary.json         | 实验配置和最佳结果                     |

这些输出让项目不只是“前端看起来能跑”，还可以用实验文件复查算法效果。

## 11.8 系统流程小结

系统完整流程可以概括为：

- 本地数据集
  - > 数据清洗和画像构建
  - > 综合评分计算
  - > 排序、搜索和推荐算法
  - > FastAPI 统一接口
  - > 前端交互展示
  - > 用户行为记录
  - > For You 动态刷新
  - > 实验输出和测试验证

这个结构把课程要求中的数据结构、排序算法、搜索算法、推荐算法和复杂度分析连接成一个完整系统。

# 十二、实验结果与算法对比总结

## 12.1 实验目标

本项目的实验不是只看某一个算法是否能跑通，而是围绕三个问题展开：

- 排序时，完整排序和 Top-N 选择哪个更适合推荐榜？
- 搜索时，线性扫描和索引搜索差距有多大？
- 推荐时，不同数据集应该使用怎样的相似度和个性化方法？

因此实验结果分为三类：

- 排序运行时间实验。
- 搜索运行时间实验。
- 推荐模型评估实验。

## 12.2 排序实验总结

MovieLens 当前共有 9,742 部电影。Netflix 当前评分表中有 17,770 部电影。

在 MovieLens 上，取 Top-10 时：

| 数据规模  | 归并排序       | 堆排序        | Top-N 堆选择  |
|-------|------------|------------|------------|
| 9,742 | 0.0309048s | 0.0488330s | 0.0005376s |

在 Netflix 上，取 Top-10 时：

| 数据规模   | 归并排序       | 堆排序        | Top-N 堆选择  |
|--------|------------|------------|------------|
| 17,770 | 0.0644513s | 0.1153770s | 0.0010930s |

实验结论是：

- 如果需要完整排行榜，可以使用归并排序或堆排序。
- 如果只需要前 10 或前 20 个推荐结果，Top-N 堆选择更合适。
- 推荐系统通常只展示少量结果，因此没有必要每次完整排序全部电影。

从复杂度角度看：

完整排序：O(n log n)  
Top-N 堆选择：O(n log k)

当 k 很小而 n 较大时，Top-N 方法优势明显。

### 12.3 搜索实验总结

MovieLens 搜索实验对比了线性搜索和索引搜索。

典型结果如下：

| 查询        | 搜索类型 | 线性搜索       | 索引搜索       | 结果数   |
|-----------|------|------------|------------|-------|
| Toy Story | 标题   | 0.0010660s | 0.0000154s | 3     |
| Comedy    | 类型   | 0.0026210s | 0.0000043s | 3,756 |
| funny     | 标签   | 0.0019635s | 0.0002299s | 25    |

可以看到，索引搜索明显快于线性搜索。尤其是类型搜索，Comedy 会命中 3,756 部电影，但索引搜索仍然很快。

搜索实验说明：

- 线性搜索适合作为算法基线，逻辑简单。
- 倒排索引更适合频繁查询。
- 模糊搜索和 n-gram 索引提高了容错能力。
- Netflix 由于没有类型和标签，只保留标题搜索能力。

### 12.4 评分实验总结

评分部分的核心结论是：不能直接使用原始平均分。

项目使用贝叶斯评分解决低样本电影偶然高分问题：

bayesian\_rating  
= (v \* R + m \* C) / (v + m)

MovieLens 综合评分为：

评分质量 70 + 热度 20 + 标签 5 + 近期活跃 5

Netflix 综合评分为：

评分质量 70 + 热度 25 + 近期活跃 5

两者差异来自数据字段：

| 数据集       | 是否有标签 | 综合评分差异       |
|-----------|-------|--------------|
| MovieLens | 有     | 增加 tag_score |
| Netflix   | 无     | 把标签权重转给热度    |

同时，项目还实现了用户宽严修正：

```
preference_adjusted_rating
    = rating - user_avg_rating + global_avg_rating
```

这让系统能够区分“严格用户给 4 分”和“宽松用户给 4 分”的不同含义。

## 12.5 推荐实验总结

MovieLens 推荐系统的重点是信息融合。

MovieLens Similar Movies 使用：

```
series_score
+ genre_overlap
+ tag_overlap
+ title_similarity
+ comprehensive_score
```

MovieLens For You 使用：

```
长期兴趣
+ 短期兴趣
+ 明确喜欢
- 明确不喜欢
+ Similar Users 协同过滤
+ Explore 高质量探索
```

Netflix 推荐系统的重点是大规模评分行为。

Netflix Similar Movies 使用：

```
目标电影作为 5 分种子
-> 用户-用户协同过滤
-> 同名系列候选强规则合并
```

Netflix For You 在线版现在使用：

```
会话行为伪评分
-> 多路候选召回
-> Weighted RRF 合并候选
-> 加载 feature_weights.csv 和 mf_model.npz
-> Stacked Linear Hybrid Reranker 重排
-> 必要时协同过滤或高分电影回退
```

Netflix 离线训练版负责产出：

- 矩阵分解
- + 多路候选召回
- + Learning-to-Rank
- + 神经重排扩展
- + 在线模型 artifact

这说明两个数据集不是简单复用同一套算法，而是根据字段和数据规模选择不同的推荐策略。

## 12.6 Netflix 离线推荐评估总结

当前 10,000 用户规模 Netflix 实验中，主要结果为：

| 算法                             | Precision@10 | Recall@10 | HitRate@10 | MAP@10   |
|--------------------------------|--------------|-----------|------------|----------|
| Popular Quality                | 0.074112     | 0.024215  | 0.307436   | 0.045325 |
| User-User CF                   | 0.029713     | 0.017375  | 0.223835   | 0.010972 |
| Matrix Factorization           | 0.008912     | 0.003754  | 0.076176   | 0.003675 |
| Hybrid MF + CF + Quality       | 0.080880     | 0.033912  | 0.385129   | 0.045146 |
| Stacked Linear Hybrid Reranker | 0.102468     | 0.051382  | 0.477188   | 0.058697 |

结论是：

- 热门质量基线很强，说明推荐系统不能忽视全局质量。
- 单独 MF 覆盖率高，但 Top-10 命中表现不稳定。
- Hybrid 模型比单独模型更好。
- Learning-to-Rank 重排进一步提升 Top-K 指标。

这与真实推荐系统经验一致：最终效果通常来自多路信号融合，而不是单一算法。

## 12.7 工程性能总结

为了让 Netflix 实验能在本地运行，项目实现了多项加速：

| 优化             | 解决的问题           |
|----------------|-----------------|
| DuckDB         | 大规模评分数据本地查询     |
| user_norms 预计算 | 避免重复计算用户向量长度    |
| 稀疏矩阵协同过滤       | 替代 Python 双重循环  |
| ANN 近似近邻       | 大用户规模下避免全量两两比较  |
| MF 批量打分        | 用矩阵乘法替代逐个预测     |
| 特征缓存           | 避免重复构造用户-候选特征   |
| 数组查找           | 替代大量 dict.get   |
| argpartition   | 只取 Top-K，避免完整排序 |

这些优化说明项目不仅实现了算法，也考虑了算法在真实数据规模下的可运行性。

# 十三、项目亮点、不足与改进方向

## 13.1 项目亮点一：算法主线完整

本项目覆盖了算法课程中多个核心主题：

| 主题    | 项目中的体现                          |
|-------|---------------------------------|
| 排序算法  | 归并排序、堆排序、Top-N 堆选择              |
| 搜索算法  | 线性搜索、倒排索引、n-gram 模糊搜索           |
| 数据结构  | 字典索引、倒排表、堆、稀疏矩阵                 |
| 相似度计算 | 标题相似度、余弦相似度、协同过滤相似度             |
| 推荐算法  | 内容推荐、协同过滤、矩阵分解、Learning-to-Rank |
| 复杂度分析 | 排序、搜索、Top-K 和候选召回复杂度            |

因此它不是单点功能，而是围绕电影推荐场景形成了一条完整算法链路。

### 13.2 项目亮点二：双数据集设计

项目同时支持 MovieLens 和 Netflix Prize。

MovieLens 适合做课堂主线：

- 数据小，启动快。
- 字段完整，有标题、类型、标签和评分。
- 适合讲搜索、标签、相似推荐和个性化推荐闭环。

Netflix 适合作为进阶扩展：

- 数据规模大。
- 评分行为丰富。
- 适合讲大规模协同过滤、矩阵分解、候选召回、重排训练和工程优化。

这种设计让项目既能满足课程作业第一版演示，也能展示更深入的推荐系统扩展能力。

### 13.3 项目亮点三：不伪造数据能力

项目对数据边界处理比较清楚。

例如 Netflix 没有 genre 和 tag，系统就明确不支持 Netflix 的类型搜索和标签搜索，而不是返回伪造结果。这一点看似简单，但对推荐系统很重要：算法应该基于真实数据字段，而不是为了功能完整而制造不存在的信息。

同样，MovieLens 有标签，所以项目对标签做了规范化、别名合并、权重计算和语义扩展；Netflix 没有标签，所以改用评分行为、矩阵分解和协同过滤。

### 13.4 项目亮点四：前后端和实验闭环

项目不仅有算法代码，还有：

- 本地前端页面。
- FastAPI 后端。
- 命令行实验入口。
- 实验输出文件。
- 单元测试和 HTTP smoke 测试。
- 一键启动脚本。

这让汇报不只是展示公式，还可以现场演示：

搜索电影

- > 查看相似推荐
- > 点击 Like / Dislike
- > For You 发生变化
- > 查看排序和搜索运行时间

算法结果可以在页面上看，也可以在 `output/` 文件中复查。

## 13.5 当前不足一：在线推荐仍是本地会话级别

当前 MovieLens 和 Netflix 的在线 For You 都是基于本地浏览器会话行为，并不是完整用户系统。

也就是说，项目没有实现：

- 用户登录。
- 长期用户账号画像。
- 多设备同步。
- 真实线上反馈闭环。

这对于课程项目是合理的，因为重点是算法和数据结构。但如果要扩展成真实产品，需要加入用户系统和长期画像存储。

## 13.6 当前不足二：Netflix 在线模型仍是轻量适配器

Netflix 在线 For You 已经接入了离线训练出的 stacked linear hybrid reranker，会读取 `summary.json`、`feature_weights.csv` 和 `mf_model.npz` 做在线重排。

不过当前接入方式仍然是轻量在线适配器，而不是完整线上训练平台。它主要复用离线训练得到的线性权重和矩阵分解向量，并在在线侧重新构造可用召回路由。

当前仍然存在几个限制：

- 在线特征与离线特征尽量对齐，但部分路由仍是在线近似实现。
- 神经重排器还没有作为在线推理模型接入。
- 还没有完整的模型版本管理、自动刷新和线上 A/B 评估机制。
- 如果 artifact 缺失，系统会回退到用户-用户协同过滤。

后续可以改进为更完整的模型服务：

离线训练

- > 模型版本管理
- > 在线特征一致性校验
- > 轻量模型服务
- > For You 稳定加载最新模型

## 13.7 当前不足三：推荐解释还可以更细

当前系统已经给出一些推荐理由，例如：

- 匹配用户喜欢的类型或标签。
- 相似用户也高分评价。
- 同系列电影。
- 高分冷启动推荐。

但解释仍然可以更细，例如：

- 显示具体是哪几个标签影响推荐。
- 显示与哪部用户喜欢的电影最相似。
- 显示协同过滤支持人数和平均评分。
- 显示推荐来自哪个召回路由。

这些解释可以让用户更容易理解“为什么推荐这部电影”。

### 13.8 当前不足四：模型评估还可以更系统

目前已有排序时间、搜索时间、Netflix Top-K 指标等实验，但还可以进一步补充：

- MovieLens For You 的离线评估。
- 不同参数下的消融实验。
- 标签语义扩展前后的效果对比。
- Similar Movies 推荐质量的人工评测。
- Netflix 不同用户规模下的训练耗时对比。

这些内容如果继续扩展，可以让项目从课程作业进一步接近完整实验报告。

### 13.9 后续改进方向

后续可以从四个方向继续优化。

第一，在线推荐模型继续升级。当前 Netflix 已经接入 stacked linear reranker，后续可以把神经重排器、模型版本管理、在线特征一致性检查和自动 artifact 刷新补上。

第二，推荐解释增强。把推荐理由从简单文案扩展成结构化解释，例如来源桶、关键标签、相似用户数量、系列匹配原因和质量分贡献。

第三，实验体系完善。为 MovieLens 和 Netflix 分别补充更多离线评估、消融实验和参数对比，让结果更有说服力。

第四，数据规模继续扩展。MovieLens 可以切换到更大的版本，Netflix 可以进一步测试全量用户下的 ANN、批量打分和缓存效果。

### 13.10 总结陈述

本项目围绕“影视流媒体用户观看行为排序与推荐系统”完成了一个可运行的算法实验平台。

在 MovieLens 上，项目完成了从电影画像、综合评分、排序搜索、相似电影推荐到 For You 个性化推荐的完整闭环；在 Netflix Prize 上，项目进一步扩展到大规模评分数据处理、协同过滤、矩阵分解、多路候选召回、Learning-to-Rank、在线重排、神经重排和训练加速。

从算法角度看，项目覆盖了排序、搜索、堆、倒排索引、相似度、协同过滤、矩阵分解和监督学习重排。从工程角度看，项目实现了前端展示、FastAPI 接口、命令行实验、数据集切换、实验输出和测试验证。

因此，这个项目既能作为算法与数据结构课程作业的完整第一版，也具备继续扩展成更深入推荐系统实验平台的基础。